

Tecnológico de Monterrey

1er Mini-Proyecto Cuántica

Luis Daniel González Alcocer A01286903@tec.mx
Andrés Saul Pérez Martínez A01199376@tec.mx
Andros Izhar Alvarado Sánchez A00842107@tec.mx

Profesor: Julio César Gutiérrez Vega
Curso: Análisis de sistemas cuánticos (Gpo 603)

19 de octubre de 2026

Índice

1. Parte 1: Análisis dinámico del pozo finito simétrico [45 puntos] **3**

2. Parte 2: Análisis dinámico del pozo finito asimétrico [50 puntos] **19**

3. Reflexion final **30**
3.1. Andros Izhar Alvarado Sánchez 30

A. Códigos de problema 1 Inciso 2 **30**

B. Códigos de problema 1 Inciso 3(a) **35**

C. Códigos de problema 1 Inciso 3(b) **39**

D. Códigos de problema 1 Inciso 4(b) **43**

E. Códigos de problema 2 Inciso 6 **45**

1. Parte 1: Análisis dinámico del pozo finito simétrico [45 puntos]

Inciso (1.)

Cálculo de las amplitudes. Aplicando la condición de normalización $\int_{-\infty}^{\infty} |\varphi_n(x)|^2 \, \mathrm{d}x = 1$, demuestren **analíticamente** que las amplitudes A_n y B_n en los eigenestados pares e impares del pozo finito simétrico están dadas por las Ecs. 18 y 19

$$A_n = \sqrt{\frac{q_n}{a q_n + 1}}, \qquad B_n = \frac{\cos k_n a}{\exp(-q_n a)} A_n, \tag{18}$$

donde $k_n = (\pi/2a)\,\mu_n$ y $q_n = (\pi/2a)\sqrt{\mu_{\text{máx}}^2 - \mu_n^2}$. Para los eigenestados impares obtenemos

$$A_n = \sqrt{\frac{k_n}{a k_n - \tan(a k_n)}}, \qquad B_n = \frac{\sin k_n a}{\exp(-q_n a)} A_n. \tag{19}$$

El pozo finito simétrico tiene la siguiente función de potencial.

$$U(x) = \begin{cases} 0, & |x| < a \\ U_0, & |x| > a \end{cases}, \qquad U_0 \in \mathbb{R} > 0.$$

Para este pozo en particular, tenemos las siguientes soluciones dependiendo del intervalo en función de k_n y q_n. Por ahora, analicemos los eigenestados pares. **Eigenestados pares** (simétricos)

$$\varphi_n(x) = \begin{cases} A_n \cos k_n x, & |x| < a, \qquad k_n = \sqrt{2ME_n/\hbar^2}, \\ B_n \exp(-q_n x), & x > a, \qquad q_n = \sqrt{2M(U_0 - E_n)/\hbar^2}. \end{cases}$$

Se inicia con la condición de normalización, que implica que la partícula debe estar en algún lugar en todo el espacio...

$$1 = \int_{-\infty}^{-a} B_n^2 \exp(2q_n x)\, dx + \int_{a}^{\infty} B_n^2 \exp(-2q_n x)\, dx + 2A_n^2 \int_0^a \cos^2(k_n x)\, dx \tag{1.1}$$

Por simetría, las primeras dos integrales de ecuación 1.1 son simétricas y juntas dan un resultado de $B_n^2/q_n \exp(-2q_n a)$, mientras que la última integral se resuelve usando la identidad $\cos^2(k_n x) = \frac{1}{2}(1 + \cos(2k_n x))$. Por lo que ecuación 1.1 se simplifica a:

$$1 = \frac{B_n^2}{q_n} \exp(-2q_n a) + A_n^2 \left(a + \frac{1}{2k_n} \sin(2ak_n) \right) \tag{1.2}$$

Para simplificar esto, se necesita una relación entre la variable B_n y A_n. Dado que φ_n debe ser continua, ambos deben valer lo mismo en $x = a$. Por lo que:

$$A_n \cos(k_n a) = B_n \exp(-q_n a) \tag{1.3}$$

Ahora se reescribe 1.2 en función de A_n y se busca resolver para A_n.

$$1 = A_n^2 \left(\frac{\cos^2(k_n a)}{q_n} + a + \frac{1}{2k_n} \sin(2ak_n) \right) \tag{1.4}$$

$$A_n^2 = \frac{1}{\frac{\cos^2(ak_n)}{q_n} + a + \frac{1}{2k_n} \sin(2ak_n)} \tag{1.5}$$

Multiplicando por q_n y utilizando la ecuación característica $\tan(ak_n) = \frac{q_n}{k_n}$, se puede llegar a la siguiente ecuación:

$$A_n^2 = \frac{q_n}{\cos^2(k_n a) + q_n a + \sin^2(k_n a)} \tag{1.6}$$

$$A_n = \sqrt{\frac{q_n}{1 + q_n a}} \tag{1.7}$$

Donde la ecuación 1.7 nos da el valor de A_n para que este sea siempre normalizado. Ahora, para obtener B_n se utiliza la ecuación 1.3 y se llega a:

$$B_n = \frac{\cos(k_n a)}{\exp(-q_n a)} A_n \tag{1.8}$$

Ahora, se continua con los estados impares.

- **Eigenestados impares** (antisimétricos)

$$\varphi_n(x) = \begin{cases} A_n \sin k_n x, & |x| < a, \quad k_n = \sqrt{2ME_n/\hbar^2}, \\ B_n \exp(-q_n x), & x > a, \quad q_n = \sqrt{2M(U_0 - E_n)/\hbar^2}. \end{cases} \tag{1.9}$$

Nota: en la fuente original las definiciones de k_n y q_n para los eigenestados impares aparecen intercambiadas. Aquí se corrigen para que coincidan con las de los eigenestados pares, ya que k_n corresponde a la región $|x| < a$ donde $U = 0$, y q_n a la región $|x| > a$ donde $U = U_0$.

Se inicia nuevamente con la condición de normalización, que lleva a:

$$\frac{1}{2} = \int_0^a A_n^2 \sin^2(k_n x)\, dx + \int_a^\infty B_n^2 \exp(-2q_n x)\, dx \tag{1.10}$$

$$\frac{1}{2} = \frac{1}{2} \left[A_n^2 \int_0^a (1 - \cos(2k_n x))\, dx + \frac{B_n^2}{q_n} \exp(-2q_n x) \Big|_\infty^a \right] \tag{1.11}$$

$$1 = A_n^2 \left[a - \frac{\sin(2k_n a)}{2k_n} \right] + \frac{B_n^2}{q_n} \exp(-2q_n a) \tag{1.12}$$

Ahora, se usa nuevamente la condición de frontera:

$$A_n \sin(k_n a) = B_n \exp(-q_n a) \tag{1.13}$$

Por lo que ahora en términos de A_n 1.12 se vuelve:

$$1 = A_n^2 \left[a - \frac{\sin(2k_n a)}{2k_n} + \frac{\sin^2(k_n a)}{q_n} \right] \tag{1.14}$$

$$A_n^2 = \frac{1}{a - \frac{\sin(2k_n a)}{2k_n} + \frac{\sin^2(k_n a)}{q_n}} \tag{1.15}$$

Para simplificar 1.15 se utiliza la ecuación maestra:

$$\cot(k_n a) = -\frac{q_n}{k_n} \tag{1.16}$$

Usando 1.16 y multiplicando por q_n 1.15, se llega a:

$$A_n^2 = \frac{q_n}{a q_n + \cos^2(k_n a) + \sin^2(k_n a)} \tag{1.17}$$

$$A_n = \sqrt{\frac{q_n}{a q_n + 1}} \tag{1.18}$$

Esta expresión es equivalente a la Ec. (19) del enunciado. De la ecuación maestra 1.16 se tiene que $1/q_n = -\tan(k_n a)/k_n$, por lo que

$$A_n^2 = \frac{1}{a + \frac{1}{q_n}} = \frac{1}{a - \frac{\tan(a k_n)}{k_n}} = \frac{k_n}{a k_n - \tan(a k_n)} \quad \Rightarrow \quad A_n = \sqrt{\frac{k_n}{a k_n - \tan(a k_n)}} \tag{1.19}$$

que es justamente la forma de la Ec. (19).
Por lo que B_n llega de 1.13:

$$B_n = \frac{\sin(k_n a)}{\exp(-q_n a)} A_n \tag{1.20}$$

Quedando entonces demostrado el cálculo de las amplitudes.

Inciso (2.)

Cálculo de las eigenenergías y estados estacionarios. Elabora una rutina computacional para encontrar las eigenenergías E_n en un pozo finito. Para fines numéricos, consideren las constantes $\hbar = 1$, semi-ancho del pozo $a = 1$, y masa $M = \pi^2/8$. Estos valores están pensados para que la energía basal del pozo infinito [Ec. (15)] sea $V_0 = 1$. Como ejemplo concreto, para la profundidad del pozo toma $U_0 = 110$.
Programa una función que evalúe los eigenestados $\varphi_n(x)$ del pozo finito.
Para el reporte:
→ Tabulen los valores de todas las energías permitidas en el pozo.
→ Grafiquen los primeros 8 eigenestados $\varphi_n(x)$ del pozo. Corroboren que entre más grande es la energía, el eigenestado penetra más en la región clásicamente *prohibida*, es decir, donde $|x| > a$. Asegúrense de usar las amplitudes A_n y B_n analíticas para que sus eigenestados

estén normalizados automáticamente. Para tener un indicio de que las cosas están correctas, verifiquen usando diversas parejas de eigenestados que se cumple la ortonormalidad

$$\int_{-\infty}^{\infty} \varphi_p^*(x)\, \varphi_q(x)\, \mathrm{d}x = \delta_{pq}. \tag{1}$$

Esta integral puede hacerse con la regla del trapecio si muestrean la coordenada x con suficiente resolución y tomando un intervalo lo suficientemente grande para asegurarse que las exponenciales han decaído prácticamente a cero. Así el error de truncamiento es despreciable.

Sobre el código:

El código se puede ver en el apéndice. Véase el Apéndice A. Ahora sí, continuemos con el problema.

Primero, se muestra en la Tabla 1 los distintos valores de las energías encontrados al resolver la ecuación maestra. Como el pozo es finito, hay un número finito de energías permisibles para estados atrapados. En este caso, se obtienen 11 eigenenergías.

n	Energías pares	n	Energías impares
1	0.8887	2	3.5531
3	7.9880	4	14.1841
5	22.1268	6	31.7943
7	43.1531	8	56.1502
9	70.6928	10	86.5879
11	103.2200	–	–

Tabla 1: Tabulación de los estados de energía para los estados tanto pares e impares. Se muestra en este caso 6 energías para los estados pares y 5 para los impares.

Asimismo, en la Figura 1, se puede ver de una forma gráfica las energías, en comparación al pozo finito. Se observa que las primeras energías están muy juntas, mientras que las últimas quedan muy cerca del borde del pozo, $U_0 = 110$.

Se espera que los eigenestados con energías bajas se parezcan a los eigenestados del pozo infinito, mientras que los eigenestados con energías altas penetren más en las paredes del pozo.

En la Figura 2, se pueden observar los primeros 8 eigenestados del pozo finito. Como se esperaba, los primeros eigenestados, se parecen al pozo infinito, dado que casi no se llega a penetrar la pared. En cambio, en los estados de mayor energía este efecto es más notorio. Ahora, lo último que queda es checar la ortonormalidad de las ecuaciones. Se realiza la siguiente integral de manera computacional:

$$\int_{-\infty}^{\infty} \varphi_p^*(x)\, \varphi_q(x)\, \mathrm{d}x = \delta_{pq}. \tag{1.21}$$

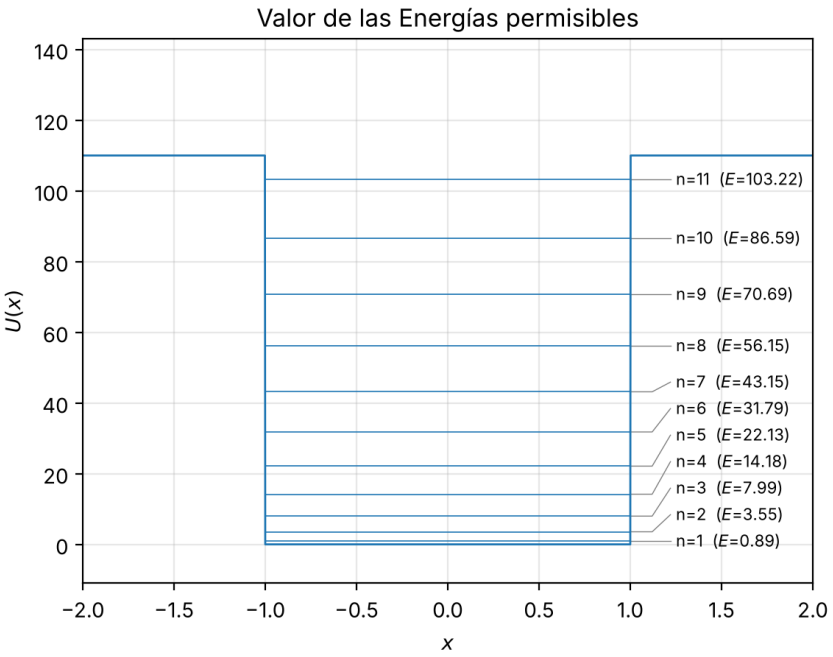


Figura 1: Se muestran como rayas todas las energías permitidas en el pozo. Para los valores específicos de $a = 1, U_0 = 110, V_0 = 1, M = \frac{\pi^2}{8}$, se obtuvieron 11 energías. Se observa que la separación entre cada energía va aumentando conforme aumenta el nivel.

Ahora, para poder ver que realmente los eigenestados son ortonormales, utilizamos una matriz de traslape S en el cual sus entradas se definen de la siguiente manera:

$$S_{ij} = \int_{-\infty}^{\infty} \varphi_i^*(x)\, \varphi_j(x)\, \mathrm{d}x \tag{1.22}$$

El resultado de S se puede apreciar en la Figura 3. Claramente aquí se demuestra la ortogonalidad dado que representa una matriz identidad, que es equivalente a la delta de Kronecker que se utiliza en la ecuación 1.21.

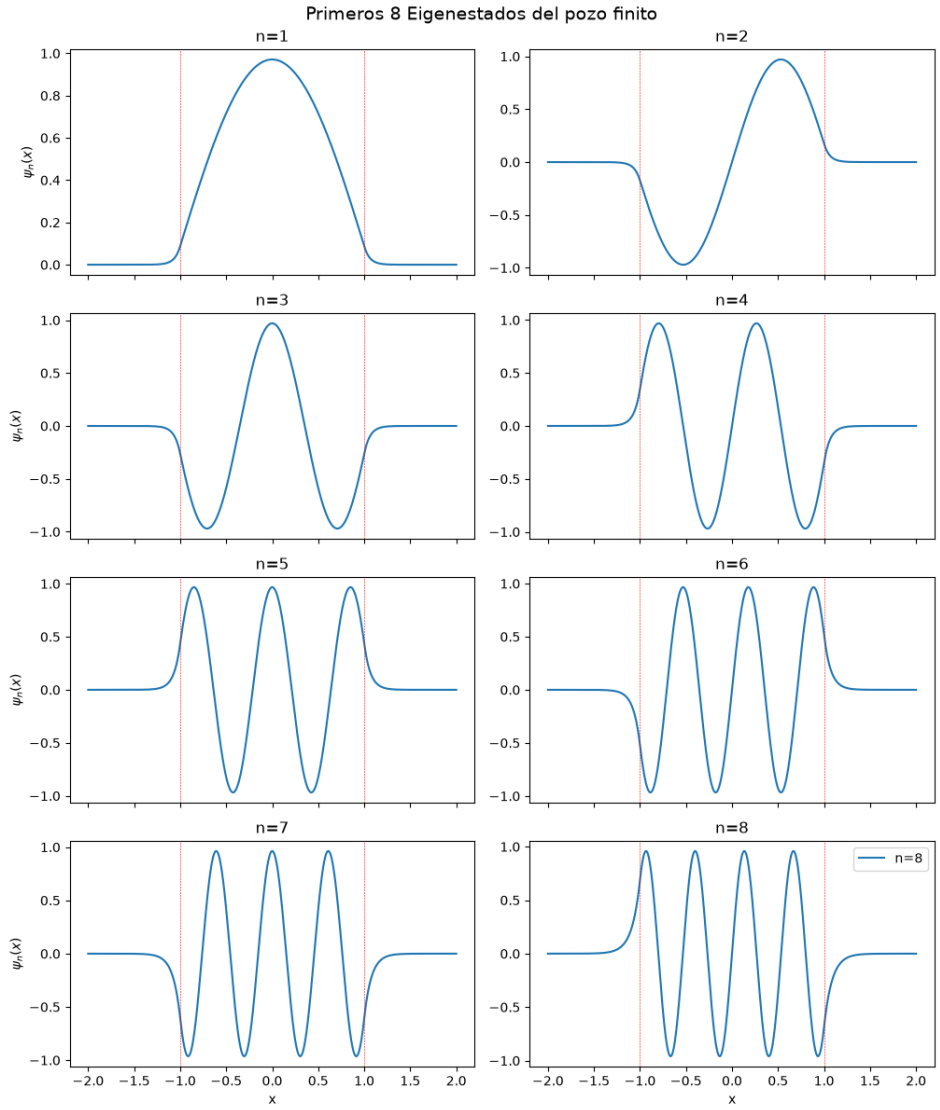


Figura 2: Los eigenestados de las primeras 8 energías del pozo. Todos están normalizados gracias a las amplitudes analíticas A_n y B_n. En la región del pozo, se muestran oscilaciones de senos o cosenos, mientras que después del pozo se observan exponenciales por la zona prohibida. Para energías altas, se llega a penetrar más en la zona prohibida.

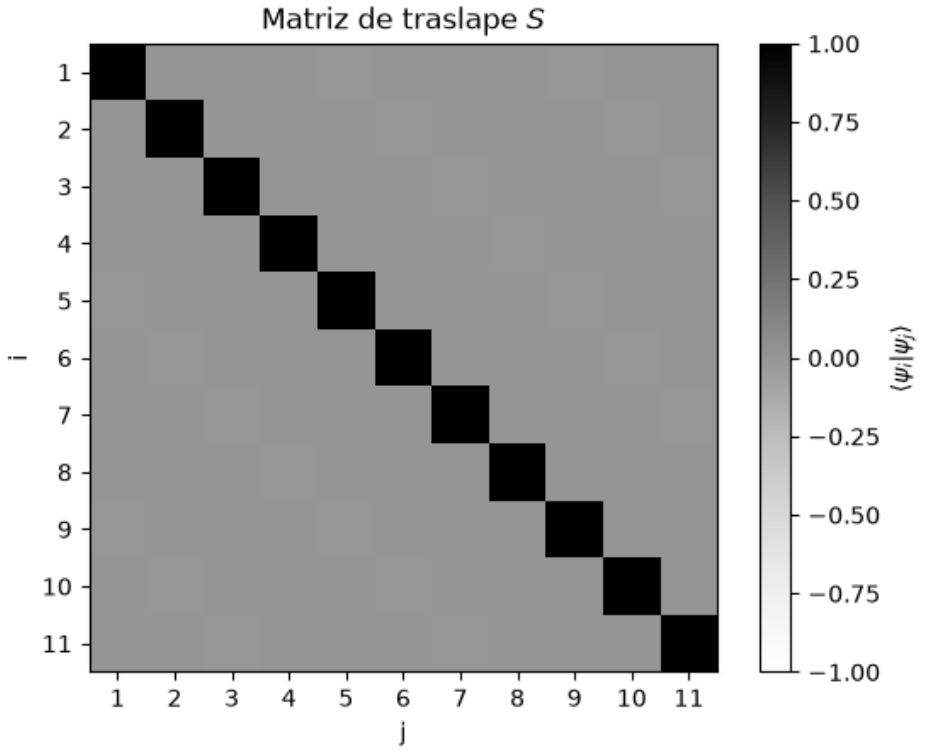


Figura 3: La matriz de traslape S que muestra la ortonormalidad de los eigenestados. Como solamente la diagonal principal contiene valores no nulos, se muestra la ortogonalidad. Asimismo, como esta tiene una diagonal de 1's, vemos que está también normalizada y por lo tanto los eigenestados son ortonormales.

Inciso (3.)

Evolución temporal. En el problema anterior hemos encontrado la base de eigenestados ortogonales del pozo finito, las funciones $\varphi_n(x)$, donde $n = \{1, 2, \ldots, N\}$ con N siendo el máximo número de eigenestados atrapados en el pozo. Programen una rutina computacional para simular la evolución temporal de una función de onda inicial $\phi(x, 0) = f(x)$ dentro del pozo finito de potencial. Como base del algoritmo, usen la expansión estándar en términos de eigenestados

$$\phi(x,t) = \sum_{n=1}^{N} c_n \varphi_n(x) \exp(-\mathrm{i}\omega_n t), \qquad \omega_n = \frac{E_n}{\hbar}, \tag{2}$$

donde $\varphi_n(x)$ es el n-ésimo estado estacionario del sistema cuya eigenenergía es E_n. Los coeficientes c_n dependen de la forma inicial de la función de onda, en general

$$c_n = \int_{-\infty}^{\infty} \varphi_n^*(x) f(x)\, \mathrm{d}x. \tag{3}$$

a) Simulen la evolución de una función de onda inicial dada por

$$\phi(x,0) = A\left[\varphi_1(x) + 0.7\varphi_2(x) - \mathrm{i}1.2\varphi_3(x) + (2-\mathrm{i})\varphi_4(x)\right], \tag{4}$$

donde A es la constante de normalización (calcúlenla previamente).

Para el reporte:

→ Generen una gráfica bidimensional de la función de densidad de probabilidad $|\phi(x,t)|^2$.

→ Grafiquen el valor esperado de la posición en función del tiempo.

b) **Pulso Gaussiano.** Consideren el pulso Gaussiano de ancho $2b$ centrado en $x = c$

$$\phi(x,0) = f(x;b,c) = A\exp\left[-\frac{(x-c)^2}{b^2}\right]\exp\left(\mathrm{i}\frac{Mv_0}{\hbar}x\right), \tag{5}$$

donde A es la constante de normalización. Consideremos que la anchura $2b$ es lo suficientemente pequeña para que la función Gaussiana prácticamente sea cero en las paredes $x = \pm a$. El pulso de la Ec. (5) tiene velocidad inicial v_0 en $t = 0$, es decir, un momentum inicial $p_0 = Mv_0$.

Para el reporte:

→ Calculen **analíticamente** la constante de normalización A.

Para las simulaciones que siguen tomen $b = 0.2$ y $c = 0.4$. Simulen la evolución del pulso de la Ec. (5) en el pozo finito. Consideren exclusivamente los N eigenestados atrapados $\varphi_n(x)$ del pozo finito para hacer la expansión.

→ Generen una gráfica bidimensional de la función de densidad de probabilidad $|\phi(x,t)|^2$.

→ Grafiquen el valor esperado de la posición en función del tiempo $\langle x \rangle$.

→ Hagan un video de la evolución del pulso. Importante: Formato mp4.

Consideraciones físicas a tomar en cuenta: Es importante notar que para expandir el pulso inicial $f(x)$ solo tenemos un número finito de eigenestados atrapados $\varphi_n(x)$ del pozo finito. En general la base finita de estados $\varphi_n(x)$ no será una base completa para expandir cualquier función arbitraria $f(x)$ inicial del pozo. Supongamos que $g(x)$ sea la función aproximada de $f(x)$ usando la base finita, $g(x) = \sum_{n=1}^{N} c_n \varphi_n(x)$. La evolución de $g(x)$ preserva energía ya que se puede descomponer de manera exacta en eigenestados del sistema. Pero esto no va a ocurrir con la función exacta $f(x)$, ya que una parte de la función no puede reconstruirse con la base finita de los estados atrapados $\varphi_n(x)$. La consecuencia física de esto es que cuando el pulso $f(x)$ evoluciona dentro del pozo una parte de su energía se disipará por las paredes como ondas viajeras que se alejan del pozo. En un tiempo largo, el pulso que quede dentro del pozo corresponderá a una función que puede construirse exclusivamente con los eigenestados $\varphi_n(x)$. La analogía de este problema en óptica corresponde a la pérdida de energía de un pulso de luz en el acoplamiento de una fibra óptica. La reconstrucción completa de la función $f(x)$ podría hacerse usando la superposición finita de estados atrapados [lo cual nos da la función $g(x)$] más una superposición continua de ondas viajeras con energía mayor a U_0 [lo cual nos da el complemento $f(x) - g(x)$].

Sobre el código:

El código del inciso a) se puede ver en el Apéndice B y el del inciso b) en el Apéndice C.

Inciso a)

Para la evolución se utilizan los $N = 11$ eigenestados del inciso 2, con las amplitudes analíticas.

Primero se calcula la constante de normalización. Escribimos la ecuación (4) como $\phi(x,0) = A\sum_{n=1}^{4} a_n \varphi_n(x)$, donde $a_1 = 1$, $a_2 = 0.7$, $a_3 = -1.2\mathrm{i}$ y $a_4 = 2 - \mathrm{i}$. Se inicia con la condición de normalización:

$$1 = \int_{-\infty}^{\infty} |\phi(x,0)|^2 \, dx = |A|^2 \sum_{p=1}^{4}\sum_{q=1}^{4} a_p^* a_q \int_{-\infty}^{\infty} \varphi_p^*(x)\varphi_q(x)\, dx \tag{1.23}$$

Por la ortonormalidad de los eigenestados [ecuación 1.21], la integral es δ_{pq}, por lo que todos los términos cruzados se anulan:

$$1 = |A|^2 \sum_{p=1}^{4}\sum_{q=1}^{4} a_p^* a_q \, \delta_{pq} = |A|^2 \sum_{n=1}^{4} |a_n|^2 \tag{1.24}$$

$$1 = |A|^2 \left(1^2 + 0.7^2 + |-1.2\mathrm{i}|^2 + |2 - \mathrm{i}|^2\right) \tag{1.25}$$

$$1 = |A|^2 \, (1 + 0.49 + 1.44 + 5) = 7.93 \, |A|^2 \tag{1.26}$$

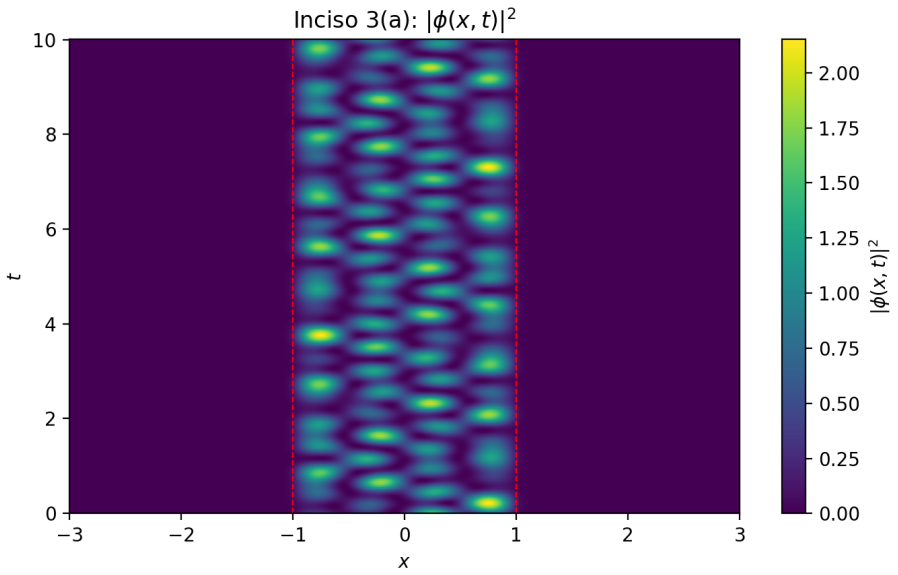


Figura 4: Densidad de probabilidad $|\phi(x,t)|^2$ para el estado inicial de la ecuación (4). Las líneas rojas marcan las paredes del pozo en $x = \pm a$.

$$\boxed{A = \frac{1}{\sqrt{7.93}} \approx 0.3551} \tag{1.27}$$

Por lo que los coeficientes de la ecuación (2) son $c_n = Aa_n$, es decir, $c_1 = A$, $c_2 = 0.7A$, $c_3 = -1.2\mathrm{i}A$, $c_4 = (2-\mathrm{i})A$ y $c_n = 0$ para $n \geq 5$. Con esto se evalúa la ecuación (2) para $t \in [0, 10]$ y el valor esperado de la posición se calcula con la regla del trapecio:

$$\langle x \rangle(t) = \frac{\int_{-\infty}^{\infty} x\, |\phi(x,t)|^2\, dx}{\int_{-\infty}^{\infty} |\phi(x,t)|^2\, dx} \tag{1.28}$$

Se divide entre la norma porque en el inciso b) la función que se evoluciona no tiene norma uno.

En la Figura 4 se muestra la densidad de probabilidad. Como solo intervienen los primeros cuatro estados, que casi no penetran las paredes, la probabilidad se queda prácticamente dentro del pozo y forma un patrón de interferencia que se mueve de un lado a otro.

Para entender el comportamiento de $\langle x \rangle$, se sustituye la ecuación (2) en el numerador de la ecuación 1.28:

$$\langle x \rangle(t) = \sum_{p,q} c_p^* c_q\, x_{pq}\, e^{-\mathrm{i}(E_q - E_p)t/\hbar}, \qquad x_{pq} = \int_{-\infty}^{\infty} \varphi_p(x)\, x\, \varphi_q(x)\, dx \tag{1.29}$$

Como φ_1 y φ_3 son pares y φ_2 y φ_4 son impares, cuando p y q tienen la misma paridad el integrando de x_{pq} es impar y $x_{pq} = 0$. Entonces solo contribuyen las parejas $(1,2)$, $(3,2)$, $(3,4)$ y $(1,4)$, y $\langle x \rangle$ oscila con las frecuencias $\omega_{pq} = (E_q - E_p)/\hbar$. Usando la Tabla 1:

$$\omega_{12} = 2.66, \qquad \omega_{32} = 4.43, \qquad \omega_{34} = 6.20, \qquad \omega_{14} = 13.30$$

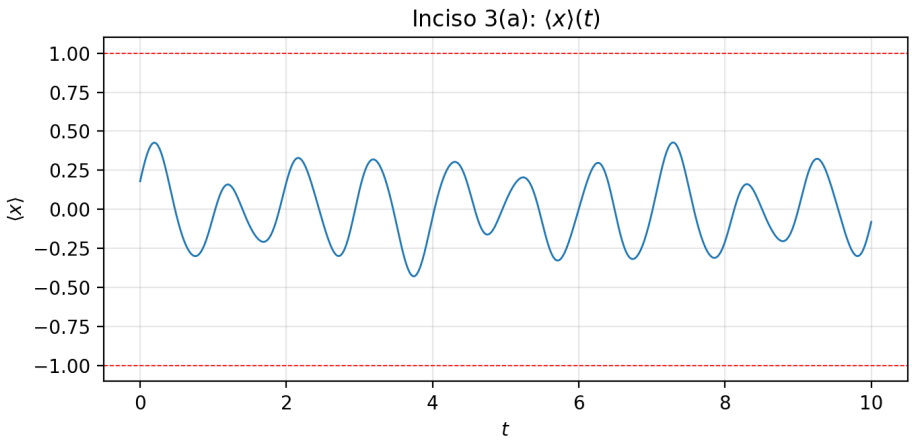


Figura 5: Valor esperado de la posición $\langle x \rangle(t)$ para el estado de la ecuación (4).

La oscilación más marcada tiene periodo $2\pi/\omega_{34} \approx 1.01$, ya que c_3 y c_4 son los coeficientes más grandes, y está modulada por las demás frecuencias, como se ve en la Figura 5.

Inciso b)

Primero se calcula la constante de normalización de manera analítica. Se inicia con la condición de normalización:

$$1 = \int_{-\infty}^{\infty} |\phi(x,0)|^2 \, dx \tag{1.30}$$

Como $|\exp(iMv_0x/\hbar)|^2 = 1$, la fase no contribuye a la norma, y elevando al cuadrado la Gaussiana se tiene:

$$1 = |A|^2 \int_{-\infty}^{\infty} \exp\left[-\frac{2(x-c)^2}{b^2}\right] dx \tag{1.31}$$

Para resolver la integral se hace el cambio de variable

$$u = \frac{\sqrt{2}}{b}(x-c), \qquad du = \frac{\sqrt{2}}{b}\,dx \quad \Rightarrow \quad dx = \frac{b}{\sqrt{2}}\,du$$

Los límites de integración no cambian, por lo que la ecuación 1.31 se vuelve:

$$1 = |A|^2 \frac{b}{\sqrt{2}} \int_{-\infty}^{\infty} e^{-u^2} \, du \tag{1.32}$$

$$1 = |A|^2 \frac{b}{\sqrt{2}} \sqrt{\pi} = |A|^2 \, b \sqrt{\frac{\pi}{2}} \tag{1.33}$$

donde se usó la integral Gaussiana $\int_{-\infty}^{\infty} e^{-u^2} du = \sqrt{\pi}$. Despejando y tomando A real y positiva:

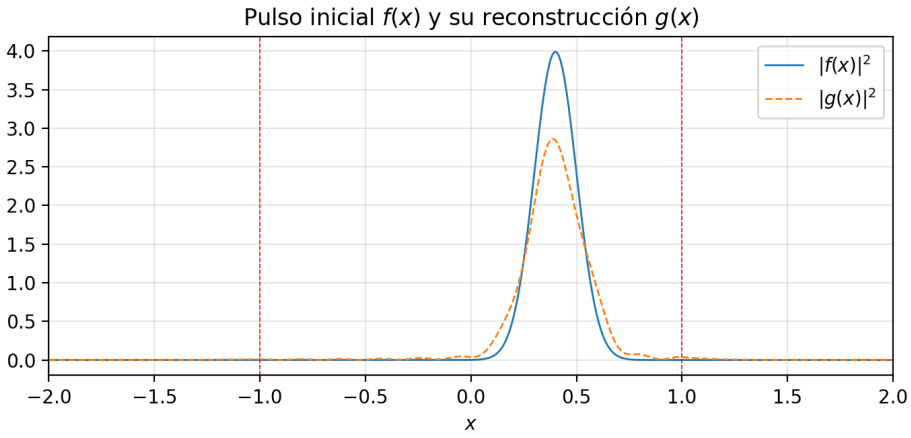


Figura 6: Pulso Gaussiano inicial $|f(x)|^2$ con $b = 0.2$, $c = 0.4$ y $v_0 = 8$, y su reconstrucción $|g(x)|^2$ con los $N = 11$ estados atrapados.

$$|A|^2 = \frac{1}{b}\sqrt{\frac{2}{\pi}} \tag{1.34}$$

$$A = \left(\frac{2}{\pi b^2}\right)^{1/4} \tag{1.35}$$

$$\boxed{A = \left(\frac{2}{\pi b^2}\right)^{1/4}} \tag{1.36}$$

Para $b = 0.2$ se obtiene $A \approx 1.997$. Se integró en toda la recta real porque, como dice el enunciado, la Gaussiana es prácticamente cero en las paredes.

El enunciado no fija v_0, por lo que se tomó $v_0 = 8$. Los coeficientes c_n se calcularon con la ecuación (3) usando la regla del trapecio. Sumando sus módulos al cuadrado se obtiene la probabilidad que se puede representar con los estados atrapados:

$$\sum_{n=1}^{11} |c_n|^2 \approx 0.910 \tag{1.37}$$

Es decir, solo el 91 % del pulso se puede construir con los estados atrapados. El 9 % restante corresponde a componentes con energía mayor a U_0, que son las que escaparían del pozo como ondas viajeras, tal como se menciona en las consideraciones físicas. En la Figura 6 se compara el pulso inicial $f(x)$ con su reconstrucción $g(x) = \sum_{n=1}^{N} c_n \varphi_n(x)$: tiene el mismo centro, pero es más baja y ancha, ya que su área es 0.910 en lugar de 1.

El pulso parte de $x = c = 0.4$ con velocidad $v_0 = 8$, por lo que se espera que llegue a la pared $x = a$ en un tiempo

$$t \approx \frac{a - c}{v_0} = \frac{1 - 0.4}{8} \approx 0.075$$

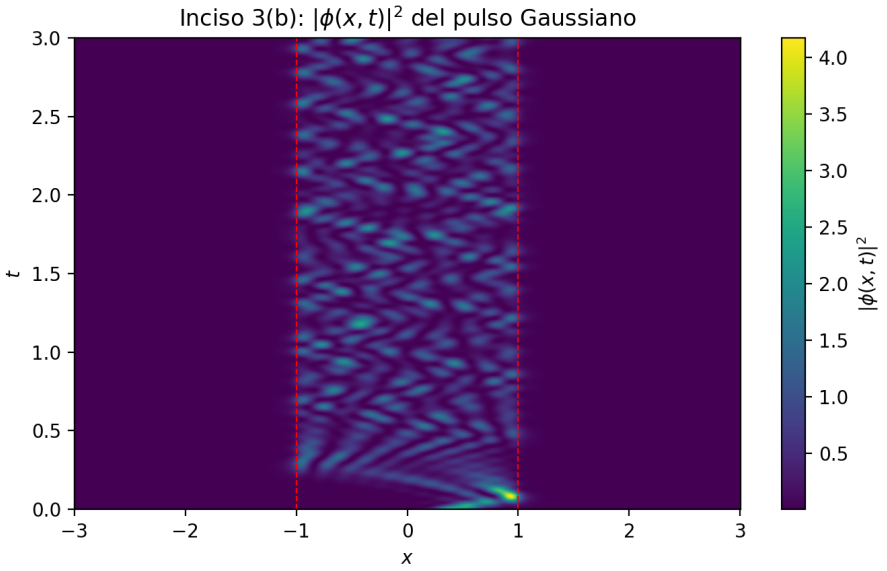


Figura 7: Densidad de probabilidad $|\phi(x,t)|^2$ del pulso Gaussiano en el pozo simétrico.

Esto coincide con la Figura 7, donde el pulso viaja hacia la derecha, llega a la pared y se refleja. Como se excitan los 11 estados, el pulso se dispersa rápido y después de unos cuantos rebotes llena todo el pozo con un patrón de interferencia.

Esto también se ve en la Figura 8: $\langle x \rangle$ sube al acercarse a la pared, rebota y después oscila cada vez menos alrededor del centro del pozo. La evolución completa se puede ver en el video `pulso_pozo_simetrico.mp4`.

Inciso (4.)

En clase analizamos el potencial escalón definido por

$$U(x) = \begin{cases} 0, & x < 0 \\ U_0, & x \geq 0 \end{cases}, \qquad U_0 \in \mathbb{R} > 0. \tag{6}$$

El estado $\phi(x)$ con energía E está dado por

$$\phi(x) = \begin{cases} A\exp(\mathrm{i}kx) + B\exp(-\mathrm{i}kx), & x \leq 0, \qquad k = \sqrt{\dfrac{2ME}{\hbar^2}}, \\ C\exp(-qx), & x > 0, \qquad q = \sqrt{\dfrac{2M}{\hbar^2}(U_0 - E)}, \end{cases} \tag{7}$$

donde A, B, C son las amplitudes de las ondas incidente, reflejada y transmitida, respectivamente. Como el estado no está atrapado, la energía E de la partícula es un parámetro libre dentro del intervalo $0 < E < U_0$. En clase encontramos que las amplitudes relativas de la

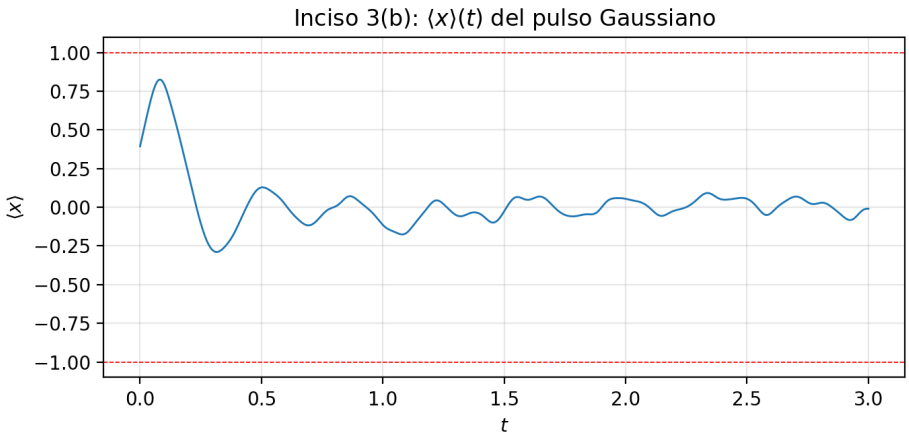


Figura 8: Valor esperado de la posición $\langle x \rangle (t)$ del pulso Gaussiano en el pozo simétrico.

onda reflejada y transmitida con respecto a la incidente son

$$\frac{B}{A} = \frac{k - \mathrm{i}q}{k + \mathrm{i}q} = \exp(\mathrm{i}\gamma), \qquad \text{donde} \qquad \gamma = -2\arctan\left(\frac{q}{k}\right), \tag{8}$$

$$\frac{C}{A} = \frac{2k}{k + \mathrm{i}q}. \tag{9}$$

La fase γ representa el desfasamiento de la onda reflejada con respecto a la incidente al momento de la reflexión en la pared.

a) Consideren ahora el pozo finito de potencial simétrico y encuentren una ecuación característica unificada (es decir, que sirva para estados pares e impares) para los eigenvalores de energía. Para esto consideren que un estado estacionario dentro del pozo resulta de la superposición de dos estados viajeros contrapropagantes de la forma $\exp(\pm \mathrm{i}kx)$. Cuando la onda incidente se refleja en una pared del pozo sufre un corrimiento de fase γ. Un ciclo completo dentro del pozo incluye la reflexión en las dos paredes y el doble tránsito de la onda a lo largo del pozo. La condición de resonancia y reproducibilidad implica que el desfase total en un ciclo completo en la cavidad sea un múltiplo entero de 2π. Pueden usar esta condición dinámica para deducir la ecuación característica unificada.

b) Evalúen la ecuación característica que obtuvieron en el inciso (a) con los datos numéricos de los incisos anteriores y demuestren que las eigenenergías son coincidentes.

Sobre el código:

El inciso a) es analítico. El código del inciso b) se puede ver en el Apéndice D.

Inciso a)

Dentro del pozo ($|x| < a$) el potencial es cero, por lo que un estado estacionario se puede escribir como la superposición de dos ondas viajeras contrapropagantes:

$$\varphi(x) = \alpha\, e^{\mathrm{i}kx} + \beta\, e^{-\mathrm{i}kx}, \qquad k = \sqrt{\frac{2ME}{\hbar^2}} \tag{1.38}$$

Cada pared del pozo es localmente un escalón de altura U_0, por lo que se puede usar el resultado de las ecuaciones (8) y (9). Midiendo la coordenada desde la pared, la onda que llega a ella se refleja con un corrimiento de fase

$$\gamma = -2\arctan\left(\frac{q}{k}\right), \qquad q = \sqrt{\frac{2M(U_0 - E)}{\hbar^2}} \tag{1.39}$$

Como el pozo es simétrico, el corrimiento de fase es el mismo en las dos paredes.

Ahora se sigue a la onda durante un ciclo completo. Parte de la pared izquierda $x = -a$ viajando hacia la derecha y recorre una distancia $2a$ hasta $x = a$, por lo que acumula una fase $2ka$. En $x = a$ se refleja y gana la fase γ. Regresa recorriendo otra vez $2a$, acumulando $2ka$, y al reflejarse en $x = -a$ gana otra vez γ. Por lo que el desfase total en un ciclo es:

$$\Delta\Phi = 2ka + \gamma + 2ka + \gamma = 4ka + 2\gamma \tag{1.40}$$

La condición de resonancia pide que después de un ciclo la onda se reproduzca a sí misma, es decir, que el desfase total sea un múltiplo entero de 2π:

$$4ka + 2\gamma = 2\pi m, \qquad m = 0, 1, 2, \ldots \tag{1.41}$$

Sustituyendo la ecuación 1.39 y dividiendo entre 2:

$$4ka - 4\arctan\left(\frac{q}{k}\right) = 2\pi m \tag{1.42}$$

$$2ka - 2\arctan\left(\frac{q}{k}\right) = \pi m \tag{1.43}$$

Como $k, q > 0$, se puede usar la identidad $\arctan(q/k) = \pi/2 - \arctan(k/q)$:

$$2ka - \pi + 2\arctan\left(\frac{k}{q}\right) = \pi m \tag{1.44}$$

$$2ka = \pi(m + 1) - 2\arctan\left(\frac{k}{q}\right) \tag{1.45}$$

Definiendo $n = m + 1$, con $n = 1, 2, 3, \ldots$:

$$2ka = n\pi - 2\arctan\left(\frac{k}{q}\right) \tag{1.46}$$

Ahora se usa la parametrización de clase $\mu = (2a/\pi)k$. Por la ecuación (13), $k^2 + q^2 = K^2$ con $\mu_{\text{máx}} = (2a/\pi)K$, por lo que

$$k = \frac{\pi}{2a}\mu, \qquad q = \frac{\pi}{2a}\sqrt{\mu_{\text{máx}}^2 - \mu^2} \quad \Rightarrow \quad \frac{k}{q} = \frac{\mu}{\sqrt{\mu_{\text{máx}}^2 - \mu^2}}$$

Si se piensa en un triángulo rectángulo con cateto opuesto μ, cateto adyacente $\sqrt{\mu_{\text{máx}}^2 - \mu^2}$ e hipotenusa $\mu_{\text{máx}}$, se ve que

$$\arctan\left(\frac{\mu}{\sqrt{\mu_{\text{máx}}^2 - \mu^2}}\right) = \arcsin\left(\frac{\mu}{\mu_{\text{máx}}}\right)$$

Sustituyendo en la ecuación 1.46, con $2ka = \pi\mu$:

$$\pi\mu = n\pi - 2\arcsin\left(\frac{\mu}{\mu_{\text{máx}}}\right) \tag{1.47}$$

Dividiendo entre π se llega a la ecuación característica unificada:

$$\boxed{\mu_n = n - \frac{2}{\pi}\arcsin\left(\frac{\mu_n}{\mu_{\text{máx}}}\right), \qquad n = 1, 2, \ldots} \tag{1.48}$$

con $E_n = V_0\mu_n^2$. Con una sola ecuación se obtienen todos los estados, y n es directamente el número cuántico del estado.

Para comprobar que esta ecuación contiene a las ecuaciones (14) y (17), se toma la tangente de $\frac{\pi}{2}\mu = \frac{n\pi}{2} - \theta$, donde $\theta = \arcsin(\mu/\mu_{\text{máx}})$:

- Si n es impar, $\tan\left(\frac{n\pi}{2} - \theta\right) = \cot\theta$, por lo que

$$\tan\left(\frac{\pi}{2}\mu\right) = \cot\theta = \frac{\sqrt{\mu_{\text{máx}}^2 - \mu^2}}{\mu},$$

  que es la ecuación (14) de los estados pares.

- Si n es par, $\tan\left(\frac{n\pi}{2} - \theta\right) = -\tan\theta$, por lo que

$$\tan\left(\frac{\pi}{2}\mu\right) = -\tan\theta = -\frac{\mu}{\sqrt{\mu_{\text{máx}}^2 - \mu^2}},$$

  que es la ecuación (17) de los estados impares.

Por lo que la ecuación 1.48 contiene a ambas: los n impares dan los estados pares y los n pares dan los impares.

Además, si $U_0 \to \infty$ entonces $\mu_{\text{máx}} \to \infty$, el arcoseno se anula y se recupera $\mu_n = n$, es decir $E_n = n^2 V_0$, que son las energías del pozo infinito. En el pozo finito el término del arcoseno hace que $\mu_n < n$, por lo que las energías son menores; esto se debe a que la onda penetra en las paredes.

Finalmente, la función $F(\mu) = \mu + \frac{2}{\pi}\arcsin(\mu/\mu_{\text{máx}})$ es creciente y vale $F(\mu_{\text{máx}}) = \mu_{\text{máx}} + 1$, por lo que hay una solución por cada entero $n < \mu_{\text{máx}} + 1$. Entonces el número de estados atrapados es

$$N = \lceil \mu_{\text{máx}} \rceil \tag{1.49}$$

que siempre es al menos uno. Para $\mu_{\text{máx}} = \sqrt{110} \approx 10.49$ se obtiene $N = 11$, igual que en la Tabla 1.

Inciso b)

Se resolvió numéricamente la ecuación 1.48 para $n = 1, 2, \ldots$ con el método de Brent (`brentq`), buscando cada raíz en el intervalo $[0, \mu_{\text{máx}}]$. Se obtuvieron $N = 11$ soluciones, igual que con las ecuaciones de paridad. En la Tabla 2 se comparan las energías obtenidas con las de la Tabla 1.

n	E_n (Ecs. 14 y 17)	E_n (Ec. unificada)	Diferencia
1	0.8887	0.8887	4.0×10^{-15}
2	3.5531	3.5531	4.5×10^{-14}
3	7.9880	7.9880	1.1×10^{-12}
4	14.1841	14.1841	5.3×10^{-13}
5	22.1268	22.1268	1.1×10^{-14}
6	31.7943	31.7943	7.1×10^{-15}
7	43.1531	43.1531	0
8	56.1502	56.1502	5.7×10^{-14}
9	70.6928	70.6928	7.8×10^{-13}
10	86.5879	86.5879	4.3×10^{-14}
11	103.2200	103.2200	7.0×10^{-12}

Tabla 2: Comparación de las eigenenergías obtenidas con las ecuaciones de paridad (14) y (17) y con la ecuación característica unificada. Las diferencias son del orden de la precisión numérica del método, por lo que las energías coinciden.

Como se observa, las once energías coinciden y la diferencia máxima es de 7.0×10^{-12}, que corresponde a la tolerancia numérica del método de búsqueda de raíces. Por lo que se demuestra que la condición de resonancia de un ciclo completo es equivalente a resolver el problema con las condiciones de frontera, con la ventaja de que no es necesario separar los estados por paridad.

2. Parte 2: Análisis dinámico del pozo finito asimétrico [50 puntos]

Consideremos ahora el pozo finito asimétrico definido por el potencial

$$U(x) = \begin{cases} U_1, & x < 0, \\ 0, & 0 \le x < L, \\ U_2, & x \ge L, \end{cases} \qquad \{U_1, U_2\} > 0, \qquad U_2 \ge U_1, \qquad c \equiv \frac{U_2}{U_1} \ge 1, \tag{10}$$

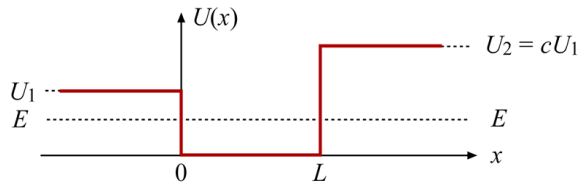


Figura 9: Pozo infinito de potencial de anchura L y paredes con potenciales diferentes

que se muestra en la Fig. 1. Sin perder generalidad, asumimos que $U_2 \geq U_1$, por lo que el factor de proporcionalidad $c \geq 1$. El factor c se considera como un parámetro de asimetría entre ambas paredes. Observen que el pozo está definido entre $x = 0$ y $x = L$, pues al no ser simétrico, ya no tiene ninguna ventaja definirlo simétricamente alrededor de $x = 0$. Definimos el inicio del pozo en $x = 0$ para facilitar evaluaciones en esa frontera.

Inciso (1.)

Escribe la forma general de las eigenfunciones $\varphi_n(x)$ del pozo asimétrico. Nota que el pozo tiene tres secciones de potencial constante. Asume que la energía de la partícula está en el rango $0 < E < U_1$. Observa que el pozo no tiene simetría, por lo que los estados $\varphi_n(x)$ no tienen paridad, como sí existía en el potencial simétrico. Esto último va a tener un impacto en la forma en la que escribas la forma del estado en la zona central de atrapamiento.

$$U_2 = cU_1, \qquad c = \frac{U_2}{U_1} \geq 1, \qquad 0 \leq E \leq U_1 \leq U_2$$

Se observa que son tres áreas con potencial finito... La onda debe cumplir la ec. de Schrödinger $\forall x$

$$-\frac{\hbar^2}{2m}\partial_x^2\varphi_+ V\varphi = E\varphi$$

Para $0 < x < L$: $E - V = E$, dado que $V = 0$.

$$\Rightarrow -\frac{\hbar^2\partial^2\varphi}{2m} = E\varphi \Rightarrow \varphi = A\cos(\omega x) + B\sin(\omega x) \quad \text{donde} \quad \omega \equiv \sqrt{\frac{2mE}{\hbar^2}}$$

Para $x < 0$: Si $U_1 - E > 0$

$$\Rightarrow \partial^2\varphi = \frac{2m}{\hbar^2}(-E + U_1)\varphi = \kappa^2\varphi, \qquad \kappa \equiv \sqrt{\frac{2m(U_1 - E)}{\hbar^2}}, \qquad \kappa > 0\ldots$$

$$\Rightarrow \varphi = C_1 e^{\kappa x} + C_2 e^{-\kappa x}$$

Claramente, el segundo término no es normalizable para $x < 0$, por lo que $C_2 = 0$. Sea $C_1 = C$

$$\Rightarrow \varphi(x) = Ce^{+\kappa x}$$

Para $x > L$: $U_2 - E > 0$. Similar al pasado sea

$$\gamma \equiv \sqrt{\frac{2m(U_2 - E)}{\hbar^2}} = \sqrt{\frac{2m(cU_1 - E)}{\hbar^2}}$$

$$\Rightarrow \varphi = De^{-\gamma x}$$

Ahora, juntamos las soluciones para formar una solución continua.

$$\varphi(x) = \begin{cases} Ce^{+\kappa x}, & x < 0, \\ A\cos(\omega x) + B\sin(\omega x), & 0 \le x < L, \\ De^{-\gamma x}, & x \ge L. \end{cases} \tag{2.1}$$

Como el potencial no es simétrico, los estados $\varphi_n(x)$ no tienen paridad definida, por lo que en la zona central se conservan ambos términos, $\cos(\omega x)$ y $\sin(\omega x)$, con A y B en general distintos de cero. Las constantes ω, κ y γ dependen de E_n.

Inciso (2.)

Encuentra la ecuación característica para las eigenenergías E_n de los estados estacionarios $\varphi_n(x)$ de una partícula atrapada en el potencial de pozo asimétrico. *Hint: Puedes intentar la estrategia del problema 4 de la parte 1 del pozo simétrico. Opcionalmente, puedes parametrizar como lo hicimos en clase para que la ecuación quede más compacta y dependa solo de un parámetro.*

Ahora las constantes A, B, C, D de $\varphi(x)$ en la ecuación 2.1 se calculan. Estas soluciones y sus derivadas deben coincidir en las condiciones de frontera . . .
En $x = 0$:

$$\boxed{C = A}, \qquad C\kappa = B\omega, \qquad \boxed{B = +\frac{\kappa}{\omega}C}$$

En $x = L$:

$$A\cos(\omega L) + B\sin(\omega L) = De^{-\gamma L}$$
$$-A\omega\sin(\omega L) + B\omega\cos(\omega L) = -D\gamma e^{-\gamma L}$$

Remplazando a A y B en términos de C . . .

$$C\cos(\omega L) + \frac{\kappa}{\omega}C\sin(\omega L) = De^{-\gamma L} \tag{2.2}$$
$$-C\omega\sin(\omega L) + \kappa C\cos(\omega L) = -D\gamma e^{-\gamma L} \tag{2.3}$$

Multiplicando por κ 2.2 y luego restando a 2.3

$$\frac{\kappa^2}{\omega}C\sin(\omega L) + C\omega\sin(\omega L) = D[\kappa + \gamma]e^{-\gamma L}$$

$$C\sin(\omega L)\left[\frac{\kappa^2}{\omega} + \omega\right] = De^{-\gamma L}[\kappa + \gamma]$$

$$\boxed{D = Ce^{\gamma L}\left[\frac{\sin(\omega L)\left[\kappa^2 + \omega^2\right]}{\omega[\gamma + \kappa]}\right]}$$

Ahora, ya tenemos todas las constantes en términos de C. Sin embargo de las ecuaciones 2.2, 2.3, podemos también obtener una ecuación que relacione las variables ω, γ, κ, lo que nos permitiría obtener los estados de energía permisibles.

Multiplicando 2.2 por γ y despejando $\frac{D}{C}e^{-\gamma L}\gamma$

$$\frac{D}{C}\gamma e^{-\gamma L} = \gamma\cos(\omega L) + \frac{\kappa}{\omega}\gamma\sin(\omega L) = -\kappa\cos(\omega L) + \omega\sin(\omega L)$$

$$\Rightarrow \gamma\cos(\omega L) + \frac{\kappa}{\omega}\gamma\sin(\omega L) = -\kappa\cos(\omega L) + \omega\sin(\omega L)$$

$$\cos(\omega L)\,[\gamma + \kappa] = \sin(\omega L)\left[\omega - \frac{\kappa\gamma}{\omega}\right]$$

$$\Rightarrow \tan(\omega L) = \frac{\gamma + \kappa}{\omega - \frac{\kappa\gamma}{\omega}} = \frac{\omega(\gamma + \kappa)}{\omega^2 - \kappa\gamma}$$

$$\boxed{\tan(\omega L) = \frac{\omega(\kappa + \gamma)}{\omega^2 - \kappa\gamma}} \qquad \leftarrow \text{Ecuación Maestra} \tag{2.4}$$

Recordando

$$\kappa = \sqrt{\frac{2m(U_1 - E)}{\hbar^2}}, \qquad \omega = \sqrt{\frac{2mE}{\hbar^2}}, \qquad \gamma = \sqrt{\frac{2m(cU_1 - E)}{\hbar^2}}$$

Notemos que

$$-\kappa^2 + \gamma^2 = \frac{2mcU_1}{\hbar^2} - \frac{2mU_1}{\hbar^2} = \frac{2mU_1}{\hbar^2}(c - 1)$$

$$\kappa^2 + \omega^2 = \frac{2mU_1}{\hbar^2}, \qquad \gamma^2 + \omega^2 = \frac{2mcU_1}{\hbar^2}$$

Usando estas ecuaciones, sea $z \equiv \omega L$ una variable adimensional

$$\Rightarrow z^2 + (\kappa L)^2 = \frac{2mU_1L^2}{\hbar^2}$$

$$\therefore z_{max} = \frac{L}{\hbar}\sqrt{2mU_1}$$

Usando z_{max}, escribimos κ y γ en términos de z

$$z^2 + (\kappa L)^2 = z_{max}^2$$

$$\Rightarrow \boxed{\kappa L = \sqrt{z_{max}^2 - z^2}}$$

$$(L\gamma^2) + (\omega L)^2 = \frac{2mcU_1L^2}{\hbar^2}$$

$$z^2 + (L\gamma)^2 = cz_{max}^2$$

$$\boxed{\Rightarrow L\gamma = \sqrt{cz_{max}^2 - z^2}}$$

Ahora, se reescribe la ecuación maestra, multiplicando y dividiendo por L^2...

$$\boxed{\tan(z) = \frac{z\sqrt{z_{max}^2 - z^2} + z\sqrt{cz_{max}^2 - z^2}}{z^2 - \sqrt{z_{max}^2 - z^2}\sqrt{cz_{max}^2 - z^2}}} \tag{2.5}$$

Inciso (3.)

Deduce una expresión cerrada para el número total N de estados estacionarios atrapados $\varphi_n(x)$ que el pozo soporta dados los parámetros U_1, U_2 y L.

Juguemos algebraicamente con la ecuación maestra (2.4), con el fin de poder encontrar otra representación apóyenos a poder comprender el ángulo z. Para esto construyamos el lado derecho como una suma de ángulo de la tangente:

$$\tan(\omega L) = \frac{\frac{\kappa}{\omega} + \frac{\gamma}{\omega}}{1 - \frac{\kappa}{\omega}\frac{\gamma}{\omega}} \tag{2.6}$$

Con esto podemos fijarnos que podemos definir las siguientes igualdades:

$$\tan(a) = \frac{\kappa}{\omega}$$
$$\tan(b) = \frac{\gamma}{\omega}$$
$$\tan(\arctan(\frac{\kappa}{\omega}) + \arctan(\frac{\gamma}{\omega})) = \frac{\arctan(\frac{\kappa}{\omega}) + \arctan(\frac{\gamma}{\omega})}{1 - \arctan(\frac{\kappa}{\omega}) * \arctan(\frac{\gamma}{\omega})}$$

De esta manera obtenemos que para la $\tan(z)$ es igual a la suma de los ángulos antes mencionados. Recordemos también la siguiente propiedad de la tangente

$$\tan(\theta) = \tan(\theta + k\pi)$$

Donde k pertenece a los enteros, por lo que esto nos dictaminara la cantidad de estados atrapados, recordemos que nosotros buscamos que nuestro contador se construya con base en empezar en 0, por lo que $k = n - 1$ para que cuando tengamos el estado 1, sea la base y k valga 0. Con esto mencionado podemos hacer el siguiente desarrollo:

$$\tan(z) = \tan(\arctan(\frac{\kappa}{\omega}) + \arctan(\frac{\gamma}{\omega})) \tag{2.7}$$
$$z = \arctan(\frac{\kappa}{\omega}) + \arctan(\frac{\gamma}{\omega}) + (n-1)\pi \tag{2.8}$$

Podemos reescribir esto en términos de z y z_max de la siguiente manera:

$$z = \arctan(\frac{\sqrt{z_{max}^2 - z^2}}{z}) + \arctan(\frac{\sqrt{cz_{max}^2 - z^2}}{z}) + (n-1)\pi \tag{2.9}$$

Con esto podemos despejar para n y obtener la función de estados atrapados:

$$n = \frac{1}{\pi}[z - \arctan(\frac{\sqrt{z_{max}^2 - z^2}}{z}) - \arctan(\frac{\sqrt{cz_{max}^2 - z^2}}{z})] + 1 \tag{2.10}$$

Pero queremos el máximo que pueda haber, para esto debemos de obtener él $z \Rightarrow z_{max}$, por lo que podemos llegar a la siguiente expresión. teniendo en cuenta algunos términos se cancelan:

$$\boxed{n = \frac{1}{\pi}[z_{max} - \arctan(\sqrt{c-1})] + 1} \tag{2.11}$$

De esta manera obtenemos una expresión del máximo de estados atrapados considerando el tope de energía con el potencial 1. Hay que aclarar que de esta función al momento de colocar valores y para que se cumpla es un entero, hay que tomar la expresión de la fracción como un redondeo hacia abajo, para respetar a n como entero.

Inciso (4.)

El pozo finito simétrico siempre soporta al menos un estado atrapado, sin importar lo estrecho o poco profundo que sea. ¿Es lo mismo para el pozo asimétrico? ¿Pueden darse las condiciones para que el pozo asimétrico no soporte ningún estado? De ser así, ¿cuál es la condición de corte para no tener estados atrapados en el pozo?

Para desarrollar esto primero tomemos el caso de tener simetría, donde hacemos que $c = 1$ por lo que veríamos que con la ecuación 2.11 se convierte en lo siguiente:

$$1 = \frac{z_{max}}{\pi} + 1$$
$$0 = \frac{z_{max}}{\pi}$$
$$0 = \frac{L\sqrt{2mU_1}}{\hbar\pi}$$

Donde entonces recordemos que la función piso actúa sobre la última fracción, lo que nos indica que si hacemos el potencial muy chico o más estrecho siempre existirá un estado atrapado, ya que esa fracción desea en mayor medida ser 0 o tener un valor entre 0 y 1, distinto a 1.
Por lo que ahora veamos para el pozo asimétrico donde c ≠ 1. Para esto podemos ver el rango de la función arcotangente, con el fin de encontrar los valores máximos y mínimos para nuestra ecuación (2.11)

$$0 < \arctan(\sqrt{c-1}) < \frac{\pi}{2}$$

Entonces podemos observar que para el caso inferior de 0 es el caso del simétrico, con esto mencionado empecemos a comparar como debe actuar la resta del z_{max} con el arcotangente. Tomemos que para disminuir el pozo o el estrecho está conectado con el valor de z_{max} por lo que incluso este valor puede ser lo suficiente chico para que el arcotangente sea mayor:

$$z_{max} < arctan(\sqrt{c-1}$$
$$z_{max} - arctan(\sqrt{c-1}) < 0$$

Al ver que estos valores son negativos siempre, podemos observar que a lo máximo y sin tocarlo esa función puede ser 0 o -$\frac{\pi}{2}$, si hacemos que z_{max} sea 0 (ya que podemos reducirlo o hacerlo lo suficiente chico tal que se acerque en el límite) y en respuesta el arcotangente está en su rango dándonos lo siguiente:

$$-\frac{\pi}{2} < z_{max} - \arctan(\sqrt{c-1} < 0$$

De manera que al dividir él π nos resulte lo siguiente:

$$-\frac{1}{2} < \frac{z_{max} - arctan(\sqrt{c-1}}{\pi} < 0$$

De esta manera podemos ahora si observar que estos valores siempre están en un rango de -0.5 y 0, por lo que si respetamos el hecho de la función piso que habíamos dicho de tomar el valor más bajo que sería el -1 (recordemos debía respetar ser un entero) podemos observar que al introducir en la ecuación 2.11 el valor de estados atrapados ($n = -1 + 1$) es igual a 0, por lo que nos es imposible decir que por más estrecho y bajo es el pozo contiene al menos un estado.
Además, podemos agregar como más específicos que la condición fuerte realizamos con respecto a variables físicas es lo siguiente:

$$0 < \frac{L}{\hbar}\sqrt{2mU_1} \leq \arctan\left(\sqrt{\frac{U_2 - U_1}{U_1}}\right) \tag{2.12}$$

Inciso (5.)

Una vez que las eigenenergías E_n han sido determinadas, las amplitudes de las funciones en cada segmento del potencial se pueden encontrar de forma algebraica cerrada (similarmente al caso de las Ecs. (18) y (19) del pozo simétrico). Usando la condición de normalización de $\varphi_n(x)$ y las conexiones entre las amplitudes de las funciones en cada segmento, determina analíticamente las amplitudes. Simplifica todo lo que se pueda.

$$\int_{-\infty}^{\infty} \varphi'(x)\varphi(x)dx = 1 \Rightarrow \frac{1}{C^2} = \int_{-\infty}^{0} e^{2\kappa x}\,dx + \int_{0}^{L}\left[\cos(\omega x) + \frac{\kappa}{\omega}\sin(\omega x)\right]^2 dx + \int_{L}^{\infty}(\beta * e^{\gamma L}e^{-\gamma x})^2 dx$$

donde

$$\beta \equiv \left[\frac{\sin(\omega L)\left[\kappa^2 + \omega^2\right]}{\omega[\gamma + \kappa]}\right]$$

$$\frac{1}{C^2} = \int_{-\infty}^{0} e^{2\kappa x}\,dx + \int_{0}^{L}\left[\cos^2(\omega x) + \frac{2\kappa}{\omega}\sin(\omega x)\cos(\omega x) + \frac{\kappa^2}{\omega^2}\sin^2(\omega x)\right]dx + \beta^2 e^{2\gamma L}\int_{L}^{\infty} e^{-2\gamma x}\,dx \tag{2.13}$$

Donde podemos reducir algunas cosas:

$$\frac{1}{C^2} = \int_{-\infty}^{0} e^{2\kappa x}\,dx + \int_{0}^{L}\left[\frac{1}{2}\left(1 + \frac{\kappa^2}{\omega^2}\right) + \frac{1}{2}\left(1 - \frac{\kappa^2}{\omega^2}\right)\cos(2\omega x) + \frac{\kappa}{\omega}\sin(2\omega x)\right]dx + \beta^2\int_{L}^{\infty} e^{-2\gamma(x-L)}\,dx$$

Integramos:

$$\Rightarrow \frac{1}{C^2} = \frac{1}{2\kappa} + \left[\frac{x}{2}\left(1 + \frac{\kappa^2}{\omega^2}\right) + \frac{\sin(2\omega x)}{4\omega}\left(1 - \frac{\kappa^2}{\omega^2}\right) - \frac{\kappa}{2\omega^2}\cos(2\omega x)\right]_0^L + \frac{\beta^2}{2\gamma}$$

$$\Rightarrow \frac{1}{C^2} = \frac{1}{2\kappa} + \frac{L}{2}\left(1 + \frac{\kappa^2}{\omega^2}\right) + \frac{\sin(2\omega L)}{4\omega}\left(1 - \frac{\kappa^2}{\omega^2}\right) + \frac{\kappa}{\omega^2}\sin^2(\omega L) + \frac{\beta^2}{2\gamma}$$

Finalmente:

$$\boxed{C = \left[\frac{1}{2\kappa} + \frac{L}{2}\left(1 + \frac{\kappa^2}{\omega^2}\right) + \frac{\sin(2\omega L)}{4\omega}\left(1 - \frac{\kappa^2}{\omega^2}\right) + \frac{\kappa}{\omega^2}\sin^2(\omega L) + \frac{\beta^2}{2\gamma}\right]^{-1/2}}$$

Inciso (6.)

Ahora toca evaluar numéricamente los resultados analíticos obtenidos anteriormente. Considera los parámetros similares al caso simétrico de la Parte 1. Es decir, $\hbar = 1$, ancho del pozo $L = 2$, y masa $M = \pi^2/8$. Para la profundidad del pozo tomen $U_1 = 85$ y $U_2 = cU_1$, donde $c \geq 1$.

a) Encuentra las energías del pozo cuando $c = 2$.

b) Grafica los eigenestados correspondientes.

c) Hagan el inciso 3(b) de la Parte 1 sobre la propagación de un pulso Gaussiano, pero ahora con el potencial asimétrico.

Sugerencia: Verifiquen que sus resultados analíticos obtenidos para el pozo asimétrico se reducen al caso del pozo simétrico cuando $c = 1$.

Primeramente, para visualizar los valores para c=2 podemos hacerlo mediante la intersección de las ecuaciones en 2.5, pero, en lugar de graficarlo para z, hagámoslo para valores de Energía. De esta manera podemos ver de mejor manera los cruces en donde se encuentran estos valores de energía que nos permiten nuestros estados. Se puede visualizar de la siguiente manera:

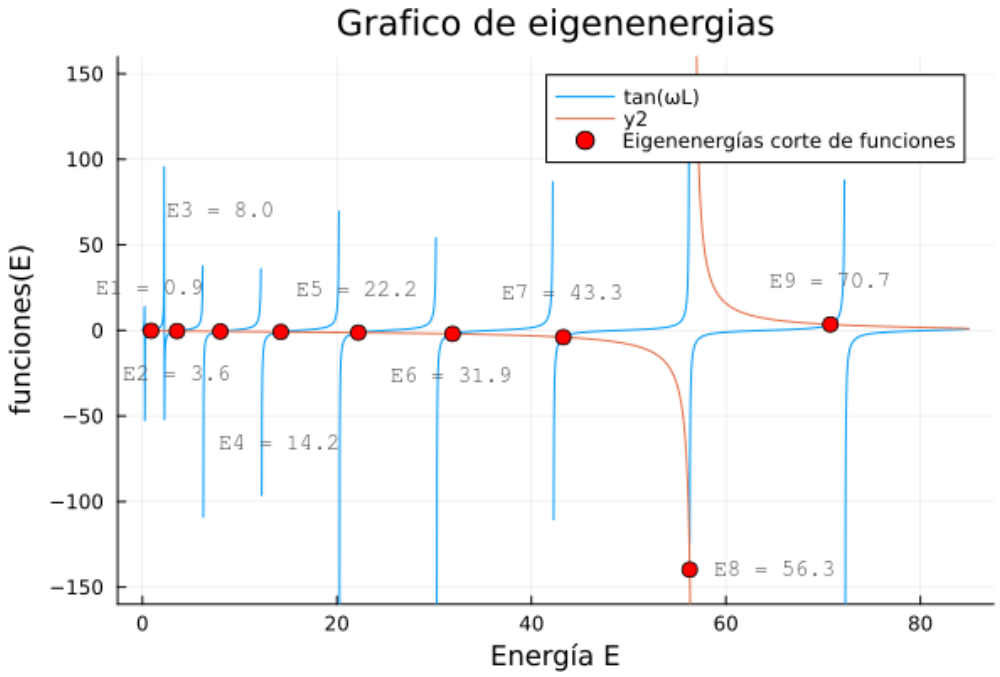


Figura 10: Gráfica de la función tangente y la función que corresponden a las energías y choques con valores de energía de cada uno

Para poder visualizar los engestados, podemos hacerlo con de manera más visual, poniendo las mismas funciones dentro del pozo. De esta manera vemos el fenómeno ondulatorio de nuestras 9 eigenenergias :

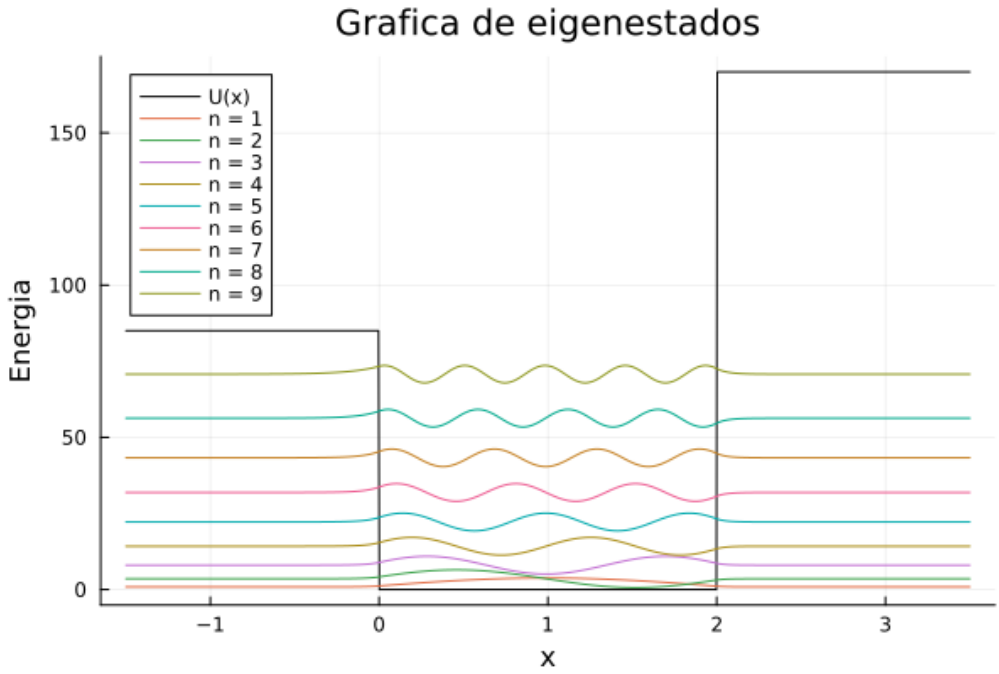


Figura 11: Gráfica de eigenestados alrededor del pozo asimétrico, con respectivo número de eigenenergia

Para este último es necesario recuperar primeramente el pulso gaussiano que se nos dio antes:

$$\phi(x,0) = f(x;b,c) = A\exp\left[-\frac{(x-c)^2}{b^2}\right]\exp\left(\mathrm{i}\frac{Mv_0}{\hbar}x\right), \tag{5}$$

Retomando entonces que el valor de A de la ecuación 1.36. Por lo que primero obtengamos los valores de c_n para nuestros estados que lo haremos su proyección en nuestra base, para poder obtenerlo como en la ecuación 3.
De esta manera obtenemos los siguientes valores:

```
c_1 = 0.43148600808995247
c_2 = 0.6199181558837233
c_3 = 0.4895196770560827
c_4 = 0.16577353090684674
c_5 = -0.13770125585852455
c_6 = -0.27244502129249015
c_7 = -0.2287758928153007
c_8 = -0.09785706249250428
c_9 = 0.01786959177148242
```

Figura 12: Valores de C encontrados de la base

De esta manera podemos también elevar al cuadrado cada uno y encontrar la probabilidad de que al tener todos los estados nos dé 1, lo cual al hacerlo numéricamente nos da 0.9930, lo cual es

muy cercano al 1, esto debido a que al no ser analítico tiene defectos.
Por lo que ahora construyamos la función de densidad de probabilidad, primero tomemos como antes g(x,t) que definimos de la siguiente manera:

$$g(x,t) = \sum c_n \varphi_n e^{-iE_n t/\hbar} \tag{2.14}$$

De esta manera podemos obtener los vectores que conforman nuestro vector, para poder obtener $g(x,t)^2$ debemos multiplicarlo por su conjugado por lo que al final modelaríamos algo como:

$$P(x,t) = \sum_{n=1}^{N} \sum_{m=1}^{N} c_n c_m^* \varphi_n(x) \varphi_m(x) \exp\left[-i\frac{(E_n - E_m)t}{\hbar}\right] \tag{2.15}$$

Por lo que ahora podemos iterar esta misma fórmula con el fin de ir construyendo con respecto al tiempo nuestra función de probabilidad, con esto podemos construir un mapa de calor donde podamos ver como cambia $g(x,t)^2$ con el tiempo y como sus valores se ven más probables de estar en ciertos momentos.

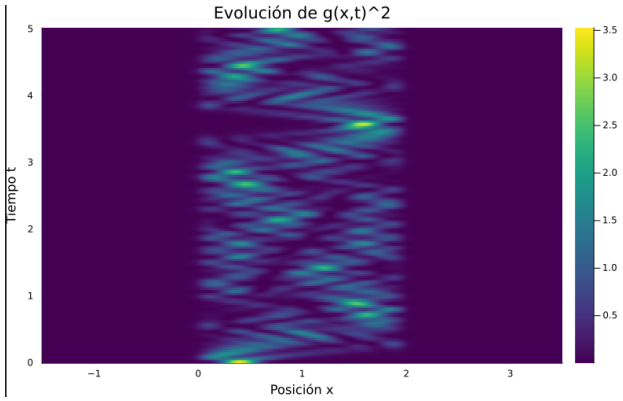


Figura 13: Mapa de calor sobre la evolución de la probabilidad en el espacio dentro del pozo también

Por lo que ahora veamos el cálculo para el primer momento, recordemos el primer momento se puede ver también de la siguiente manera:

$$< x > (t) = \int xP(x,t)dx$$

Por lo que podemos reutilizar el cálculo anterior de nuestra $g(x,t)^2$ para poder construir el primer momento y con su variación en el tiempo, resultando en la siguiente gráfica:



Figura 14: Evolución temporal del valor esperado de x en 5 segundos

Por lo que de esta manera obtenemos los momentos donde el valor esperado o el promedio de donde está la partícula contenida suele tener variaciones gracias a las energías que contiene, pero si podemos apreciar que se cumple que el valor inicial es el centro de nuestro pulso gaussiano en el tiempo 0.

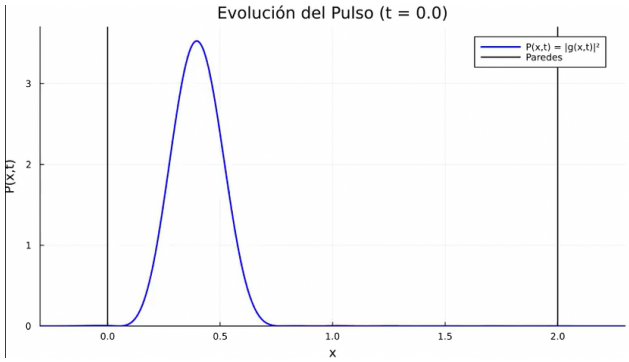


Figura 15: Captura del video del cambio de la probabilidad variando con respecto al tiempo

Con esto finalmente podemos ver como la probabilidad de la partícula varía con el tiempo, siendo que inicialmente podemos ver nuestro pulso gaussiano. Es importante mencionar consideraciones físicas, como que nuestra gaussiana como se puede ver no tiene la altura suficiente para llegar a 4, esto se debe a la construcción de los 9 estados, es insuficiente a comparación de analíticamente donde si se pueda ver la representación completa.

3. Reflexion final

3.1. Andros Izhar Alvarado Sánchez

Al comparar los dos pozos, me llamó la atención cómo la simetría se refleja en sus estados y en su evolución. En el simétrico, los estados tienen paridad definida; en el asimétrico, las paredes diferentes hacen que la función de onda se comporte distinto a cada lado. Con los parámetros del proyecto encontramos 11 estados atrapados en el primer caso y 9 en el segundo.

Lo que más me sirvió fue ver que las gráficas tenían sentido a partir de lo que habíamos planteado en el modelo. Al programarlo hubo que revisar varias veces que los resultados fueran razonables; eso me hizo fijarme más en cómo pequeños cambios en el potencial pueden cambiar lo que vemos en la simulación.

A. Códigos de problema 1 Inciso 2

Se utilizó Python y se usó la IA mayormente para resolver dudas y graficar de mejor manera. Todo el código y lógica fue hecho por los autores y estos se hacen responsables de su veracidad.

```python
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator

def energias_permisibles(mumax: float,*,eps=1e-12,a=1,hbar=1,M=1/8 * np.pi**2)
    -> np.ndarray:
    '''
    Se calcula las energías permisibles para el pozo finito. Se sabe que la
       función tan(pi/2 * x),
    tiene polos en los valores pares de x y siempre es positivo en el
       intervalo 2n, 2n+1.

    Args:
        mumax:      the maximum value of mumax. (sqrt(U0/V0))
        eps:        a small tolerance parameter
        a:          width of the finite well
        hbar:       the value of Planck reduced constant
        M:          the mass of the particle
    returns:
        Tuple(array,array) with even and odd energies respectively.

    '''
    f = lambda x: x*np.sin(np.pi*x/2) - np.sqrt(mumax**2 - x**2)*np.cos(np.pi*
       x/2)
    g = lambda x:  np.sqrt(mumax**2 - x**2)*np.sin(np.pi*x/2) +x*np.cos(np.pi*
       x/2)
    raicespar = []
    raicesimpar = []
    n = 0
    while 2*n < mumax:
        #estados pares
        low = 2*n + 1e-12
```

```
        high = min(2*n + 1-eps, mumax)
        if f(low)*f(high) < 0:
            raicespar.append(brentq(f, low, high))
        #estados impares
        low+=1; high= min(high+1,mumax)
        if low>mumax:
            n+=1
            break
        if g(low)*g(high) < 0:
            raicesimpar.append(brentq(g,low,high))
        n += 1
    E1 = (hbar*np.pi)**2/(2*M*(2*a)**2)
    x_par, x_imp = np.array(raicespar),np.array(raicesimpar)
    energias = (E1*x_par**2, E1*x_imp**2)
    return energias




def plot_energies(energias,*,a=1,ax=None):
    '''
    Plots allowed energie levels in horizontal lines.

    Args:
        energias:       array(s) that contain the energies
        a:              width of the well
        ax:             if included, the axis for graph
    Raises:
        Value error if energies is empty.
    '''
    energias =  np.sort(np.concatenate(energias))
    if len(energias) ==0:
        raise ValueError("Energías must at least have 1 value.")
    if ax is None:
        _, ax = plt.subplots()

    for (index,e) in enumerate(energias,start=1):
        ax.text(1, e, f"  n={index}", va="center_baseline")
    for e in energias:
        plt.hlines(e,-a,a,lw=0.6)
    plt.ylabel("Energía")
    ax = graph_well(ax)
    plt.title("Valor de las Energías permisibles")
    plt.show()

def graph_well(ax,*,a=1,U0=110):
    def potential(x: np.ndarray, *, a: float, U0: float) -> np.ndarray:
        """
        Helper function that returns 0 for |x| <= a and U0 otherwise.

        Args:
            x: Positions at which to evaluate the potential.
            a: Half-width of the well.
```

```
            U0: Potential height outside the well.

        Returns:
            Array of potential values with the same shape as x.
        """
        return np.where(np.abs(x) <= a, 0.0, U0)
    steps = 2001
    x = np.linspace(-2 * a, 2 * a, steps)
    y = potential(x, a=a, U0=U0)

    ax.set_xlim(-2 * a, 2 * a)
    ax.set_ylim(-0.1 * U0, 1.3 * U0)
    ax.set_xlabel(r"$x$")
    ax.set_ylabel(r"$U(x)$")
    ax.grid(alpha=0.3)
    ax.plot(x,y,lw=1)
    return ax

def plot_eigenstates(energias,M,hbar,U0,a,*, n=8):
    '''
    Function that plots eigenstates in a 4x2 grid. To graph, it also
       calculates the eigenstates.

    Args:
        energias:   tuple that contains even and odd energies in arrays.
        M:          mass of the particle
        hbar:       value of Planck reduced constant
        U0:         value of the potential outside the well
        a:          width of the well
        n:          number of eigenstates to graph (max 8)

    Raises:
        ValueError if there is not enough energies or n>8
    '''

    if (len(energias[1]) < n/2):
        raise ValueError("No hay suficientes energías.")

    if (n>8):
        raise ValueError(f"El valor de $n$ es muy grande. El máximo es 8")
    knpar = np.sqrt(2*M*energias[0])/hbar
    qnpar = np.sqrt(2*M*(U0-energias[0]))/hbar
    knimpar = np.sqrt(2*M*energias[1])/hbar
    qnimpar = np.sqrt(2*M*(U0-energias[1]))/hbar
    Anpar = np.sqrt(qnpar/(a*qnpar+1))
    Bnpar = np.cos(knpar*a)/np.exp(-qnpar*a)*Anpar
    Animpar = np.sqrt(knimpar/(a*knimpar-np.tan(a*knimpar)))
    Bnimpar = np.sin(knimpar*a) / np.exp(-qnimpar*a) * Animpar
    steps= 2001
    fig, axes = plt.subplots(4, 2, figsize=(10, 12), sharex=True)
    x1 = np.linspace(-2*a,-a,steps)
    x2 = np.linspace(-a,a,steps)
    x3 = np.linspace(a,2*a,steps)
    count = 0
```

```
    for count,ax in zip(range(n),axes.flat):
        i = count // 2
        if count % 2 == 0:
            # ESTADO PAR
            psi = np.concatenate([Bnpar[i]*np.exp(qnpar[i]*x1),
                                  Anpar[i]*np.cos(knpar[i]*x2),
                                  Bnpar[i]*np.exp(-qnpar[i]*x3)])
        else:
            # ESTADO IMPAR
            psi = np.concatenate([-Bnimpar[i]*np.exp(qnimpar[i]*x1),
                                  Animpar[i]*np.sin(knimpar[i]*x2),
                                  Bnimpar[i]*np.exp(-qnimpar[i]*x3)])
        ax.plot(np.concatenate([x1, x2, x3]), psi, label=f"n={count+1}")
        ax.set_title(f"n={count+1}")
        # well line
        ax.axvline(-a, color="red", lw=0.4, ls="--")
        ax.axvline(a, color="red", lw=0.4, ls="--")
    fig.suptitle(f"Primeros {n} Eigenestados del pozo finito", fontsize=14)
    fig.tight_layout()
    for ax in axes[-1]:
        ax.set_xlabel("x")
    for ax in axes[:, 0]:
        ax.set_ylabel(r"$\psi_n(x)$")

    plt.legend()
    plt.show()


def eigenestados(
    energias: tuple[np.ndarray, np.ndarray],
    M: float,
    hbar: float,
    U0: float,
    a: float,
    *,
    n: int = 8,
    x_ext: float = 3,
    steps: int = 6001,
) -> tuple[np.ndarray, np.ndarray]:
    """
    Computes eigenstate for a finite potential well and returns them in an
       increasing energy order.

    Args:
        energias:   tuple that contains even and odd energies in arrays.
        M:          mass of the particle
        hbar:       value of Planck reduced constant
        U0:         value of the potential outside the well
        a:          width of the well
        n:          number of eigenstates to graph (max 8)
        x_ext:      the máximum value of exterior that the eigenstates takes
        steps:      the total amount of points evaluated between -x_ext and
           x_ext

```

```
    Returns:
        A tuple x,psi where x is an array of length steps and psi are the
            eigenstates in a shape (n,steps).

    Raises:
        ValueError: If there are not enough energies for n states, or if
            any energy lies outside (0, U0).
    """
    E_par, E_imp = (np.asarray(e, dtype=float) for e in energias)
    if len(E_par) < (n + 1) // 2 or len(E_imp) < n // 2:
        raise ValueError("No hay suficientes energías.")
    if np.any((E_par <= 0) | (E_par >= U0)) or np.any((E_imp <= 0) | (E_imp >=
        U0)):
        raise ValueError("Las energías deben cumplir 0 < E < U0.")

    x = np.linspace(-x_ext * a, x_ext * a, steps)
    dentro = np.abs(x) <= a
    psi = np.empty((n, steps))

    for j in range(n):
        i = j // 2
        E = E_par[i] if j % 2 == 0 else E_imp[i]
        k = np.sqrt(2 * M * E) / hbar
        q = np.sqrt(2 * M * (U0 - E)) / hbar
        A = np.sqrt(q / (a * q + 1))
        if j % 2 == 0:
            B = A * np.cos(k * a) * np.exp(q * a)
            psi[j] = np.where(dentro, A * np.cos(k * x), B * np.exp(-q * np.
                abs(x)))
        else:
            B = A * np.sin(k * a) * np.exp(q * a)
            psi[j] = np.where(dentro, A * np.sin(k * x),
                              np.sign(x) * B * np.exp(-q * np.abs(x)))

    return x, psi

def is_ortonormal(energias,M,hbar,U0,a,*,n=11):
    '''
    Function that shows a plot of S, the overlap matrix, that is used in
        quantum mechanics to measure
    the orthogonality of the functions. The S_ij element is given by the
        internal product of psi_i and psi_j.

    Args:
        energias:   tuple that contains even and odd energies in arrays.
        M:          mass of the particle
        hbar:       value of Planck reduced constant
        U0:         value of the potential outside the well
        a:          width of the well
        n:          number of eigenstates to includei n matrix


    '''
    x, psi = eigenestados(energias,M,hbar,U0,a,n=n)
```

```
    S = np.trapezoid(psi[:, None, :] * psi[None, :, :], x, axis=-1)
    fig, ax = plt.subplots()
    im = ax.imshow(S, cmap="Greys", vmin=-1, vmax=1)
    fig.colorbar(im, ax=ax, label=r"$\langle\psi_i|\psi_j\rangle$")
    ax.set_xticks(range(len(S)), labels=range(1, len(S)+1))
    ax.set_yticks(range(len(S)), labels=range(1, len(S)+1))
    ax.set_xlabel("j")
    ax.set_ylabel("i")
    ax.set_title(f"Matriz de traslape $S$")
    plt.show()


# DATA:
hbar = 1
a = 1
M = 1/8 * np.pi**2
V0 = (hbar*np.pi)**2/(2*M*(2*a)**2)
U0 = 110
mumax= np.sqrt(U0/V0)

# RESULTS:
energias = energias_permisibles(mumax)
plot_energies(energias)
plot_eigenstates(energias,M,hbar,U0,a)
is_ortonormal(energias,M,hbar,U0,a)
```

Listing 1: Problema 1 Inciso 2.

B. Códigos de problema 1 Inciso 3(a)

```
from pathlib import Path
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt

# Compatibilidad: np.trapezoid (numpy >= 2.0) o np.trapz (versiones anteriores)
trapezoid = getattr(np, "trapezoid", None) or np.trapz

# Carpeta donde se guardan las figuras (miniproyecto1-cuantica/figuras)
FIG = Path(__file__).resolve().parent.parent / "figuras"
FIG.mkdir(exist_ok=True)

# ============================================================================

# DATA (mismos valores que el inciso 2)
# ============================================================================

hbar = 1
a = 1
```

```python
M = 1/8 * np.pi**2
V0 = (hbar*np.pi)**2/(2*M*(2*a)**2)
U0 = 110
mumax = np.sqrt(U0/V0)


#
    ================================================================================

# Funciones del inciso 2
#
    ================================================================================

def energias_permisibles(mumax: float, *, eps=1e-12, a=1, hbar=1, M=1/8*np.pi
    **2):
    '''
    Calcula las energías permisibles del pozo finito simétrico resolviendo las
    ecuaciones (14) y (17). Los estados pares tienen mu en (2n, 2n+1) y los
    impares en (2n+1, 2n+2).

    Returns:
        Tuple(array, array) con las energías pares e impares respectivamente.
    '''
    f = lambda x: x*np.sin(np.pi*x/2) - np.sqrt(mumax**2 - x**2)*np.cos(np.pi*
        x/2)
    g = lambda x: np.sqrt(mumax**2 - x**2)*np.sin(np.pi*x/2) + x*np.cos(np.pi*
        x/2)
    raicespar, raicesimpar = [], []
    n = 0
    while 2*n < mumax:
        low, high = 2*n + eps, min(2*n + 1 - eps, mumax)
        if f(low)*f(high) < 0:
            raicespar.append(brentq(f, low, high))
        low += 1; high = min(high + 1, mumax)
        if low > mumax:
            break
        if g(low)*g(high) < 0:
            raicesimpar.append(brentq(g, low, high))
        n += 1
    E1 = (hbar*np.pi)**2/(2*M*(2*a)**2)
    return E1*np.array(raicespar)**2, E1*np.array(raicesimpar)**2


def eigenestados(energias, M, hbar, U0, a, *, n=8, x_ext=3, steps=6001):
    '''
    Evalúa los primeros n eigenestados normalizados (amplitudes analíticas)
    ordenados por energía creciente.

    Returns:
        x (steps,), psi (n, steps), E (n,)
    '''
    E_par, E_imp = energias
    x = np.linspace(-x_ext*a, x_ext*a, steps)
    dentro = np.abs(x) <= a
```

```
    psi = np.empty((n, steps))
    E = np.empty(n)
    for j in range(n):
        i = j // 2
        E[j] = E_par[i] if j % 2 == 0 else E_imp[i]
        k = np.sqrt(2*M*E[j])/hbar
        q = np.sqrt(2*M*(U0 - E[j]))/hbar
        A = np.sqrt(q/(a*q + 1))
        if j % 2 == 0:
            B = A*np.cos(k*a)*np.exp(q*a)
            psi[j] = np.where(dentro, A*np.cos(k*x), B*np.exp(-q*np.abs(x)))
        else:
            B = A*np.sin(k*a)*np.exp(q*a)
            psi[j] = np.where(dentro, A*np.sin(k*x), np.sign(x)*B*np.exp(-q*np
                .abs(x)))
    return x, psi, E


#
    ============================================================================

# Funciones del inciso 3
#
    ============================================================================

def evolucion(psi, cn, E, t, *, hbar=1):
    '''
    phi(x,t) = sum_n c_n phi_n(x) exp(-i E_n t / hbar)  (Ec. 2).

    Returns:
        Arreglo complejo de forma (len(t), len(x)).
    '''
    phi = np.zeros((len(t), psi.shape[1]), dtype=complex)
    for n in range(len(E)):
        if cn[n] != 0:
            phi += cn[n]*np.exp(-1j*E[n]*t/hbar)[:, None]*psi[n][None, :]
    return phi


def valor_esperado_x(x, phi):
    '''
    <x>(t) = int x |phi|^2 dx / int |phi|^2 dx
    '''
    rho = np.abs(phi)**2
    return trapezoid(x*rho, x, axis=1)/trapezoid(rho, x, axis=1)


def mapa_densidad(x, t, phi, titulo, archivo, *, a=1):
    '''
    Gráfica bidimensional de |phi(x,t)|^2.
    '''
    fig, ax = plt.subplots(figsize=(7, 4.5))
    im = ax.pcolormesh(x, t, np.abs(phi)**2, shading="auto", cmap="viridis")
    fig.colorbar(im, ax=ax, label=r"$|\phi(x,t)|^2$")
```

```
    ax.axvline(-a, color="red", lw=0.8, ls="--")
    ax.axvline(a, color="red", lw=0.8, ls="--")
    ax.set_xlabel(r"$x$")
    ax.set_ylabel(r"$t$")
    ax.set_title(titulo)
    fig.tight_layout()
    fig.savefig(archivo, dpi=200)
    plt.show()


def grafica_x(t, xm, titulo, archivo, *, a=1):
    '''
    Gráfica del valor esperado de la posición en función del tiempo.
    '''
    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(t, xm, lw=1)
    ax.axhline(-a, color="red", lw=0.6, ls="--")
    ax.axhline(a, color="red", lw=0.6, ls="--")
    ax.set_xlabel(r"$t$")
    ax.set_ylabel(r"$\langle x\rangle$")
    ax.set_title(titulo)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(archivo, dpi=200)
    plt.show()


#
    ============================================================================

# RESULTS: inciso 3(a)
#
    ============================================================================

if __name__ == "__main__":
    energias = energias_permisibles(mumax)
    N = len(energias[0]) + len(energias[1])          # N = 11
    x, psi, E = eigenestados(energias, M, hbar, U0, a, n=N)

    # Coeficientes de la Ec. (4) y constante de normalización
    coef = np.array([1, 0.7, -1.2j, 2 - 1j])
    A = 1/np.sqrt(np.sum(np.abs(coef)**2))          # A = 1/sqrt(7.93)
    cn = np.zeros(N, dtype=complex)
    cn[:4] = A*coef

    t = np.linspace(0, 10, 1201)
    phi = evolucion(psi, cn, E, t)

    print(f"A = {A:.4f}")
    print(f"Norma en t=0: {trapezoid(np.abs(phi[0])**2, x):.10f}")

    mapa_densidad(x, t, phi, r"Inciso 3(a): $|\phi(x,t)|^2$", FIG / "
       p1_3a_mapa.png")
```

```
    grafica_x(t, valor_esperado_x(x, phi), r"Inciso 3(a): $\langle x\rangle(t)$", FIG / "p1_3a_x.png")
```

Listing 2: Problema 1 Inciso 3(a).

C. Códigos de problema 1 Inciso 3(b)

```
from pathlib import Path
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, FFMpegWriter

trapezoid = getattr(np, "trapezoid", None) or np.trapz

FIG = Path(__file__).resolve().parent.parent / "figuras"
FIG.mkdir(exist_ok=True)
VID = Path(__file__).resolve().parent.parent / "videos"
VID.mkdir(exist_ok=True)

try:
    import imageio_ffmpeg
    plt.rcParams["animation.ffmpeg_path"] = imageio_ffmpeg.get_ffmpeg_exe()
except ImportError:
    pass

hbar = 1
a = 1
M = 1/8 * np.pi**2
V0 = (hbar*np.pi)**2/(2*M*(2*a)**2)
U0 = 110
mumax = np.sqrt(U0/V0)


def energias_permisibles(mumax: float, *, eps=1e-12, a=1, hbar=1, M=1/8*np.pi**2):
    '''
    Calcula las energías permisibles del pozo finito simétrico resolviendo las
    ecuaciones (14) y (17).

    Returns:
        Tuple(array, array) con las energías pares e impares respectivamente.
    '''
    f = lambda x: x*np.sin(np.pi*x/2) - np.sqrt(mumax**2 - x**2)*np.cos(np.pi*x/2)
    g = lambda x: np.sqrt(mumax**2 - x**2)*np.sin(np.pi*x/2) + x*np.cos(np.pi*x/2)
    raicespar, raicesimpar = [], []
    n = 0
    while 2*n < mumax:
        low, high = 2*n + eps, min(2*n + 1 - eps, mumax)
        if f(low)*f(high) < 0:
```

```
            raicespar.append(brentq(f, low, high))
        low += 1; high = min(high + 1, mumax)
        if low > mumax:
            break
        if g(low)*g(high) < 0:
            raicesimpar.append(brentq(g, low, high))
        n += 1
    E1 = (hbar*np.pi)**2/(2*M*(2*a)**2)
    return E1*np.array(raicespar)**2, E1*np.array(raicesimpar)**2


def eigenestados(energias, M, hbar, U0, a, *, n=8, x_ext=3, steps=6001):
    '''
    Evalúa los primeros n eigenestados normalizados ordenados por energía.

    Returns:
        x (steps,), psi (n, steps), E (n,)
    '''
    E_par, E_imp = energias
    x = np.linspace(-x_ext*a, x_ext*a, steps)
    dentro = np.abs(x) <= a
    psi = np.empty((n, steps))
    E = np.empty(n)
    for j in range(n):
        i = j // 2
        E[j] = E_par[i] if j % 2 == 0 else E_imp[i]
        k = np.sqrt(2*M*E[j])/hbar
        q = np.sqrt(2*M*(U0 - E[j]))/hbar
        A = np.sqrt(q/(a*q + 1))
        if j % 2 == 0:
            B = A*np.cos(k*a)*np.exp(q*a)
            psi[j] = np.where(dentro, A*np.cos(k*x), B*np.exp(-q*np.abs(x)))
        else:
            B = A*np.sin(k*a)*np.exp(q*a)
            psi[j] = np.where(dentro, A*np.sin(k*x), np.sign(x)*B*np.exp(-q*np
                .abs(x)))
    return x, psi, E


def evolucion(psi, cn, E, t, *, hbar=1):
    '''
    phi(x,t) = sum_n c_n phi_n(x) exp(-i E_n t / hbar)  (Ec. 2).
    '''
    phi = np.zeros((len(t), psi.shape[1]), dtype=complex)
    for n in range(len(E)):
        if cn[n] != 0:
            phi += cn[n]*np.exp(-1j*E[n]*t/hbar)[:, None]*psi[n][None, :]
    return phi


def valor_esperado_x(x, phi):
    '''
    <x>(t) = int x |phi|^2 dx / int |phi|^2 dx
    '''
```

```
    rho = np.abs(phi)**2
    return trapezoid(x*rho, x, axis=1)/trapezoid(rho, x, axis=1)


def coeficientes(x, psi, f):
    '''
    c_n = int phi_n^*(x) f(x) dx  (Ec. 3) con la regla del trapecio.
    '''
    return np.array([trapezoid(np.conj(p)*f, x) for p in psi])


def gaussiano(x, b, c, v0, *, M=1/8*np.pi**2, hbar=1):
    '''
    Pulso Gaussiano de la Ec. (5) con A = (2/(pi b^2))^(1/4).
    '''
    A = (2/(np.pi*b**2))**0.25
    return A*np.exp(-(x - c)**2/b**2)*np.exp(1j*M*v0*x/hbar)


def mapa_densidad(x, t, phi, titulo, archivo, *, a=1):
    '''
    Gráfica bidimensional de |phi(x,t)|^2.
    '''
    fig, ax = plt.subplots(figsize=(7, 4.5))
    im = ax.pcolormesh(x, t, np.abs(phi)**2, shading="auto", cmap="viridis")
    fig.colorbar(im, ax=ax, label=r"$|\phi(x,t)|^2$")
    ax.axvline(-a, color="red", lw=0.8, ls="--")
    ax.axvline(a, color="red", lw=0.8, ls="--")
    ax.set_xlabel(r"$x$")
    ax.set_ylabel(r"$t$")
    ax.set_title(titulo)
    fig.tight_layout()
    fig.savefig(archivo, dpi=200)
    plt.show()


def grafica_x(t, xm, titulo, archivo, *, a=1):
    '''
    Gráfica del valor esperado de la posición en función del tiempo.
    '''
    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(t, xm, lw=1)
    ax.axhline(-a, color="red", lw=0.6, ls="--")
    ax.axhline(a, color="red", lw=0.6, ls="--")
    ax.set_xlabel(r"$t$")
    ax.set_ylabel(r"$\langle x\rangle$")
    ax.set_title(titulo)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(archivo, dpi=200)
    plt.show()


def grafica_fg(x, f, g, archivo, *, a=1):
```

```
    '''
    Compara el pulso inicial f(x) con su reconstrucción g(x) usando los N
        estados.
    '''
    fig, ax = plt.subplots(figsize=(7, 3.5))
    ax.plot(x, np.abs(f)**2, lw=1, label=r"$|f(x)|^2$")
    ax.plot(x, np.abs(g)**2, lw=1, ls="--", label=r"$|g(x)|^2$")
    ax.axvline(-a, color="red", lw=0.6, ls="--")
    ax.axvline(a, color="red", lw=0.6, ls="--")
    ax.set_xlim(-2*a, 2*a)
    ax.set_xlabel(r"$x$")
    ax.set_title(r"Pulso inicial $f(x)$ y su reconstrucción $g(x)$")
    ax.legend()
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(archivo, dpi=200)
    plt.show()


def video(x, t, phi, archivo, *, a=1, U0=110, fps=30, titulo=""):
    '''
    Video mp4 de la evolución de |phi(x,t)|^2 junto con el potencial.
    '''
    rho = np.abs(phi)**2
    ymax = 1.1*rho.max()
    fig, ax = plt.subplots(figsize=(8, 4.5))
    pot = np.where(np.abs(x) <= a, 0, U0)/U0*ymax*0.9
    ax.fill_between(x, 0, pot, color="red", alpha=0.12, label=r"$U(x)$ (
        escalado)")
    linea, = ax.plot(x, rho[0], lw=1.2, label=r"$|\phi(x,t)|^2$")
    re, = ax.plot(x, np.real(phi[0]), lw=0.6, alpha=0.6, label=r"Re$\,\phi$")
    ax.set_xlim(x[0], x[-1])
    ax.set_ylim(-np.sqrt(ymax), ymax)
    ax.set_xlabel(r"$x$")
    ax.legend(loc="upper right")
    ax.grid(alpha=0.3)
    texto = ax.set_title("")

    def update(i):
        linea.set_ydata(rho[i])
        re.set_ydata(np.real(phi[i]))
        texto.set_text(f"{titulo}   t = {t[i]:.3f}")
        return linea, re, texto

    anim = FuncAnimation(fig, update, frames=len(t), blit=False)
    anim.save(archivo, writer=FFMpegWriter(fps=fps, bitrate=2400))
    plt.close(fig)


if __name__ == "__main__":
    energias = energias_permisibles(mumax)
    N = len(energias[0]) + len(energias[1])
    x, psi, E = eigenestados(energias, M, hbar, U0, a, n=N)

```

```
    b, c, v0 = 0.2, 0.4, 8
    f = gaussiano(x, b, c, v0)
    cn = coeficientes(x, psi, f)
    g = sum(cn[n]*psi[n] for n in range(N))
    P = np.sum(np.abs(cn)**2)

    print(f"A = {(2/(np.pi*b**2))**0.25:.4f}")
    print(f"Norma de f: {trapezoid(np.abs(f)**2, x):.6f}")
    print("|c_n|^2 =", np.round(np.abs(cn)**2, 4))
    print(f"Suma |c_n|^2 = {P:.4f}")
    print(f"<E> de g = {np.sum(np.abs(cn)**2*E)/P:.2f}")

    grafica_fg(x, f, g, FIG / "p1_3b_fg.png")

    t = np.linspace(0, 3, 1201)
    phi = evolucion(psi, cn, E, t)
    mapa_densidad(x, t, phi, r"Inciso 3(b): $|\phi(x,t)|^2$ del pulso
        Gaussiano", FIG / "p1_3b_mapa.png")
    grafica_x(t, valor_esperado_x(x, phi), r"Inciso 3(b): $\langle x\rangle(t)
        $ del pulso Gaussiano", FIG / "p1_3b_x.png")

    print("Generando video (tarda alrededor de un minuto)...")
    t_v = np.linspace(0, 3, 451)
    sel = np.abs(x) <= 2*a
    video(x[sel], t_v, evolucion(psi[:, sel], cn, E, t_v), VID / "
        pulso_pozo_simetrico.mp4",
           titulo="Pozo simétrico")
    print("Video guardado en videos/pulso_pozo_simetrico.mp4")
```

Listing 3: Problema 1 Inciso 3(b).

D. Códigos de problema 1 Inciso 4(b)

```
import numpy as np
from scipy.optimize import brentq

hbar = 1
a = 1
M = 1/8 * np.pi**2
V0 = (hbar*np.pi)**2/(2*M*(2*a)**2)
U0 = 110
mumax = np.sqrt(U0/V0)


def energias_permisibles(mumax: float, *, eps=1e-12, a=1, hbar=1, M=1/8*np.pi
    **2):
    '''
    Calcula las energías permisibles del pozo finito simétrico resolviendo las
    ecuaciones (14) y (17).

    Returns:
        Tuple(array, array) con las energías pares e impares respectivamente.
```

```
    '''
    f = lambda x: x*np.sin(np.pi*x/2) - np.sqrt(mumax**2 - x**2)*np.cos(np.pi*x/2)
    g = lambda x: np.sqrt(mumax**2 - x**2)*np.sin(np.pi*x/2) + x*np.cos(np.pi*x/2)
    raicespar, raicesimpar = [], []
    n = 0
    while 2*n < mumax:
        low, high = 2*n + eps, min(2*n + 1 - eps, mumax)
        if f(low)*f(high) < 0:
            raicespar.append(brentq(f, low, high))
        low += 1; high = min(high + 1, mumax)
        if low > mumax:
            break
        if g(low)*g(high) < 0:
            raicesimpar.append(brentq(g, low, high))
        n += 1
    E1 = (hbar*np.pi)**2/(2*M*(2*a)**2)
    return E1*np.array(raicespar)**2, E1*np.array(raicesimpar)**2


def energias_unificadas(mumax: float, *, V0=1):
    '''
    Resuelve la ecuación característica unificada
        mu = n - (2/pi) arcsin(mu/mumax),   n = 1, 2, ...
    Hay una raíz por cada n mientras F(mumax) = mumax + 1 > n.

    Returns:
        Array con las energías E_n = V0 mu_n^2 en orden creciente.
    '''
    F = lambda mu, n: mu + (2/np.pi)*np.arcsin(mu/mumax) - n
    mus = []
    n = 1
    while F(mumax, n) > 0:
        mus.append(brentq(F, 0, mumax, args=(n,), xtol=1e-14))
        n += 1
    return V0*np.array(mus)**2


if __name__ == "__main__":
    E_par, E_imp = energias_permisibles(mumax)
    E_paridad = np.sort(np.concatenate((E_par, E_imp)))
    E_unif = energias_unificadas(mumax, V0=V0)

    print(f"N (paridad) = {len(E_paridad)},  N (unificada) = {len(E_unif)},  ceil(mumax) = {int(np.ceil(mumax))}")
    print(f"{'n':>3} {'Ecs. 14 y 17':>14} {'Unificada':>12} {'Diferencia':>12}")
    for n, (e1, e2) in enumerate(zip(E_paridad, E_unif), start=1):
        print(f"{n:>3} {e1:>14.4f} {e2:>12.4f} {abs(e1 - e2):>12.1e}")
    print(f"Diferencia máxima: {np.max(np.abs(E_paridad - E_unif)):.1e}")
```

Listing 4: Problema 1 Inciso 4(b).

E. Códigos de problema 2 Inciso 6

Se utilizó Julia y se usó la IA mayormente para resolver dudas y graficar de mejor manera. Todo el código y lógica fue hecho por los autores y estos se hacen responsables de su veracidad.

```
h = 1
L = 2
M = (pi^2)/8
U1 = 85
c = 2
U2 = c*U1
E = range(0.001, 84.999, length=3000)

Tangente = tan.((L/h)*sqrt.(2*M*E))
F2 = (sqrt.(E) .* (sqrt.(U1 .- E) .+ sqrt.(U2 .- E))) ./ (E .- sqrt.((U1 .- E) .* (U2 .- E)))

for i in 1:(length(Tangente)-1) #quiero quitar los valores que se rompen en la tangente
    if abs(Tangente[i+1] - Tangente[i]) > 160
        Tangente[i] = NaN
    end
end
for i in 1:(length(F2)-1) #quitamos lineas horizontales, ya que no representan algo
    if abs(F2[i+1] - F2[i]) > 20
        F2[i] = NaN
    end
end
g(e) = (L/h)*sqrt(2*M*e) - atan(sqrt(U1-e)/sqrt(e)) - atan(sqrt(U2-e)/sqrt(e)) #tomamos la fecuacion de 2.8
E_puntos = Float64[]
for n in 1:9 #encontramos que puntos corresponden
    f(e) = g(e) - (n - 1) * pi
    raiz = find_zero(f, (0.001, 84.999))
    push!(E_puntos, raiz)
end
Y_puntos = tan.((L/h) .* sqrt.(2*M .* E_puntos))# Evaluamos
grafica = plot(E, Tangente, ylims=(-160,160), label="tan( L )", xlabel="Energía E", ylabel="funciones(E)", title="Grafico de eigenenergias")
plot!(grafica,E,F2)
scatter!(E_puntos, Y_puntos, color=:red, markersize=5, label="Eigenenergías corte de funciones")
offsets_y = [25.0, -25.0, 70.0, -65.0, 25.0, -25.0, 25.0, 0.0, 25.0]

for i in 1:9
    e_val = round(E_puntos[i], digits=1)

    if Y_puntos[i] < -100
        annotate!(grafica, E_puntos[i] + 2.5, Y_puntos[i], text("E$i = $e_val", 7, :left, :bold))
    else
        annotate!(grafica, E_puntos[i], Y_puntos[i] + offsets_y[i], text("E$i = $e_val", 7, :center, :bold))
```

```
    end
end

display(grafica)
x = range(-1.5, L + 1.5, length=1000) #hagamos nuestros valores de x a
   graficar
region1 = x .< 0.0
region2 = (x .>= 0.0) .& (x .<= L)
region3 = x .> L
function phi_n(x, E_n, region1, region2, region3; h=1.0, L=2.0, M=(pi^2)/8.0,
   U1=85.0, U2=170.0)
    wn = sqrt(2 * M * E_n) / h
    k  = sqrt(2 * M * (U1 - E_n)) / h
    y  = sqrt(2 * M * (U2 - E_n)) / h
    beta = (sin(wn * L) * (k^2 + wn^2)) / (wn * (y + k))
    Calrevez = (1 / (2 * k)) + (L / 2) * (1 + (k^2 / wn^2)) + (sin(2 * wn * L)
        / (4 * wn)) * (1 - (k^2 / wn^2)) + (k / wn^2) * (sin(wn * L)^2) + (
       beta^2 / (2 * y))
    C = 1 / sqrt(Calrevez)
    phi = zeros(length(x))
    phi[region1] .= C .* exp.(k .* x[region1])
    phi[region2] .= C .* (cos.(wn .* x[region2]) .+ (k / wn) .* sin.(wn .* x[
       region2]))
    phi[region3] .= C .* beta .* exp.(-y .* (x[region3] .- L))
    return phi
end
U = zeros(length(x))
U[region1] .= U1
U[region2] .= 0
U[region3] .= U2
graficas = plot(x,U,color=:black,label="U(x)",xlabel = "x",ylabel = "Energia",
   title = "Grafica de eigenestados",size(850,600))
for (n, E_n) in enumerate(E_puntos)
    phi_vec =phi_n(x, E_n, region1, region2, region3)
    plot!(graficas, x, E_n .+ 3* phi_vec, label="n = $n")
end
display(graficas)
b = 0.2
cancho = 0.4
v0 = 0.0
A = (2.0 / (pi * b^2))^(0.25)

phi_0 = A .* exp.(-((x .- cancho).^2) ./ (b^2)) .* exp.(1im .* (M * v0 / h) .*
    x)
dx = x[2] - x[1]
N_estados = length(E_puntos)
c_n = zeros(ComplexF64,N_estados)
for n in 1:length(E_puntos)
    phi_vec = phi_n(x, E_puntos[n], region1, region2, region3)
    c_n[n] = sum(phi_vec.*phi_0)*dx
end
for n in 1:length(c_n)
    println("c_$n = ", real.(c_n[n]))
end
```

```
suma = 0
for or in 1:length(c_n)
    suma += (c_n[or]).^2
end
real(suma)
t_final = 5.0
t = range(0.0, t_final, length=200)
Nx = length(x)
Nt = length(t)
P_xt = zeros(Float64, Nx, Nt)
phi_m = zeros(Float64, Nx, N_estados)
for n in 1:N_estados
    phi_m[:, n] = phi_n(x, E_puntos[n], region1, region2, region3)
end
for j in 1:Nt
    tj = t[j]
    for k in 1:Nx
        xk = x[k]
        sumas = 0.0 + 0.0im

        for n in 1:N_estados
            phi_n_x = phi_m[k, n]
            cn = c_n[n]
            En = E_puntos[n]

            for m in 1:N_estados
                phi_m_x = phi_m[k, m]
                cm_conj = conj(c_n[m])
                Em = E_puntos[m]
                prob = cn * cm_conj * phi_n_x * phi_m_x * exp(-1im * (En - Em)
                    * tj / h)
                sumas += prob
            end
        end
        P_xt[k, j] = real(sumas)
    end
end
grafica_2D = heatmap(x, t, P_xt',xlabel = "Posición x", ylabel = "Tiempo t",
   title = "Evolución de g(x,t)^2",color = :viridis,size = (800, 500))

display(grafica_2D)
valor_esperado = zeros(Float64, Nt)
for j in 1:Nt
    P_columna = P_xt[:, j]
    valor_esperado[j] = sum(x .* P_columna) * dx
end
valoresperadografica = plot(t, valor_esperado,linewidth = 2,color = :blue,
   xlabel = "Tiempo t",ylabel = "<x>(t)",title = "Valor esperado de x en el
   tiempo",size = (800, 400))
display(valoresperadografica)

Tope = maximum(P_xt)
animacion = @animate for j in 1:Nt
    tj = t[j]
```

```
    P_actual = P_xt[:, j]
    plot(
        x, P_actual,
        linewidth = 2,
        color = :blue,  label = "P(x,t) = |g(x,t)|  ",xlabel = "x",ylabel = "P
            (x,t)",title = "Evolución del Pulso (t = $(round(tj, digits=2)))",
            xlims = (-0.3, L + 0.3),ylims = (0.0, Tope * 1.05),legend = :
            topright,size = (800, 450))
    vline!([0.0], color = :black, linestyle = :solid, linewidth = 1.5, label =
        "Paredes")
    vline!([L], color = :black, linestyle = :solid, linewidth = 1.5, label =
       "")
end
mp4(animacion, "pulso.mp4", fps = 15)
```

Listing 5: Problema 2 Inciso 6